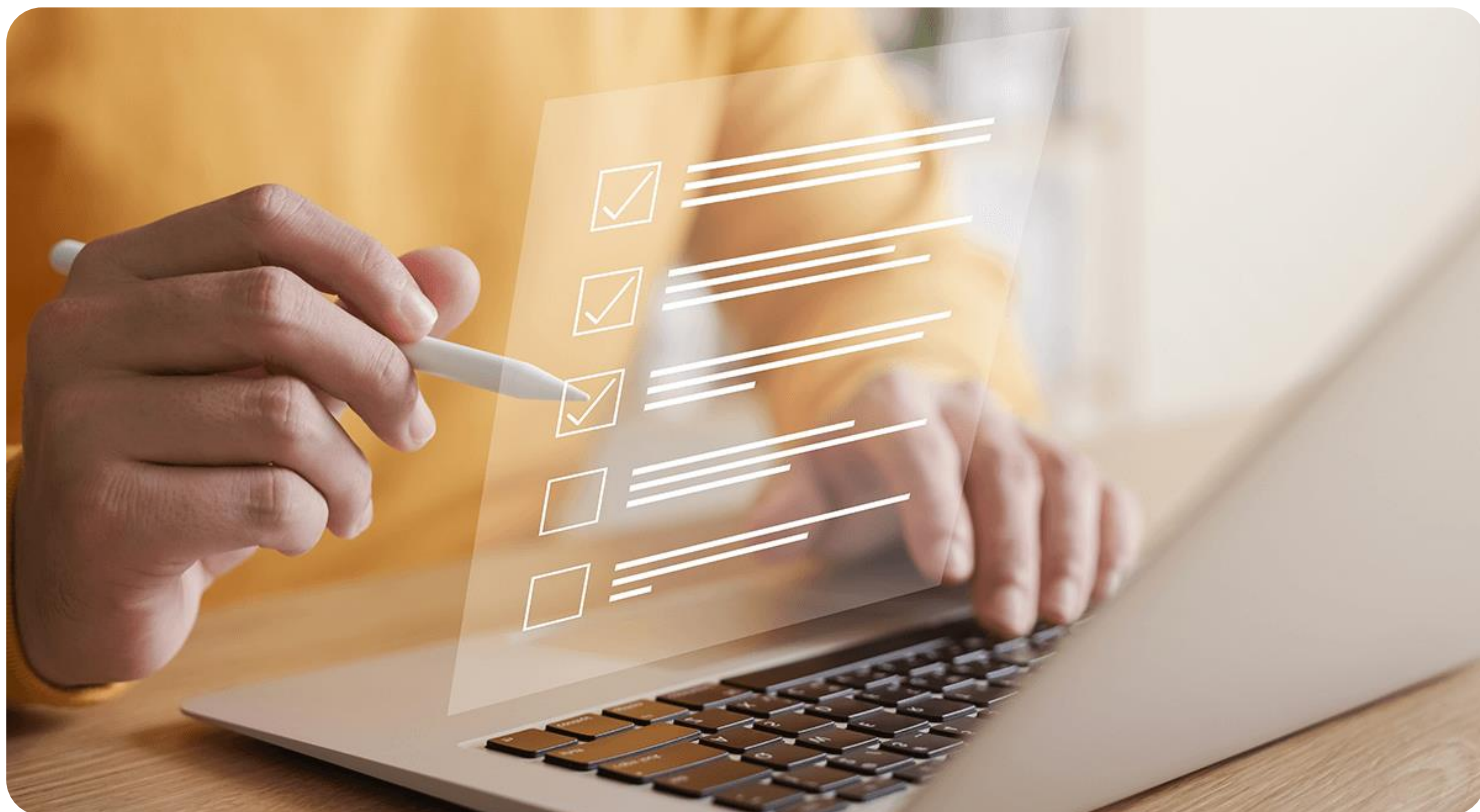




Semana 8

Ingeniería de software II

Nombre estudiantes:	Jorge Germán Gallardo Heck
	Pedro Andrés Breit Lira
Carrera: Analista Programador Computacional	
Profesor: Felipe Andrés Pérez Cifuentes	Fecha: 05/10/2026

Introducción

El presente documento corresponde al Documento de Arquitectura de Software (DAS) del sistema de gestión de pagos de gastos comunes, desarrollado en el contexto del edificio Nuevos Horizontes. Su propósito es describir de forma estructurada las decisiones de diseño, los componentes y las distintas vistas arquitectónicas que conforman la solución tecnológica propuesta para automatizar la gestión de cobros, reducir la morosidad y entregar mayor transparencia a los residentes y a la administración del edificio.

Este informe está dirigido a desarrolladores, arquitectos de software, analistas y evaluadores académicos, y busca servir como referencia técnica durante el desarrollo e implementación del sistema. El documento se organiza siguiendo el modelo de vistas 4+1, abordando los requisitos arquitectónicos, la vista de escenarios, la vista lógica, la vista de procesos, la vista de desarrollo y la vista física, para finalizar con las decisiones y patrones arquitectónicos adoptados. Su alcance se limita al diseño de la arquitectura de software, sin profundizar en aspectos de gestión de proyecto ni en el detalle de implementación del código fuente.

Presentación del informe

1. Propósito del documento

El presente Documento de Arquitectura de Software (DAS) tiene como propósito describir y documentar la arquitectura del sistema de gestión de pagos de gastos comunes desarrollado para el edificio Nuevos Horizontes. Este documento actúa como guía técnica y de referencia para el equipo de desarrollo, orientando las decisiones de diseño, la organización del código y la infraestructura sobre la cual se implementará y evolucionará el sistema a lo largo de su ciclo de vida.

Contexto del proyecto

Este documento se enmarca en un proyecto académico, desarrollado en el contexto de la asignatura Ingeniería de Software II de la carrera Analista Programador Computacional de Duoc UC. El sistema aborda una problemática real y representativa del ámbito de la administración de edificios: la gestión manual e ineficiente del cobro de gastos comunes, que genera altos niveles de morosidad, falta de transparencia hacia los residentes y sobrecarga administrativa. La solución propuesta busca alinearse con los requerimientos del negocio descritos en el caso "Nuevos Horizontes", automatizando el proceso de cobro, cálculo de cuotas y generación de informes.

Audiencia del documento

- **Desarrolladores**, quienes utilizarán este documento como referencia para implementar los distintos módulos del sistema.
- **Arquitectos de software**, como respaldo de las decisiones de diseño adoptadas.
- **Evaluadores académicos (docentes)**, quienes revisarán la coherencia y completitud del diseño arquitectónico propuesto.
- **Analistas**, que podrán utilizar este documento para validar que la solución responde a los requerimientos funcionales y no funcionales del negocio.

Alcance del documento

Este documento abarca la arquitectura de software del sistema de gestión de pagos de gastos comunes, incluyendo sus vistas de escenarios, lógica, procesos, desarrollo y física, además de las decisiones arquitectónicas y patrones de diseño adoptados. No incluye el detalle de implementación a nivel de código fuente, ni aspectos de gestión de proyecto, presupuesto o cronograma, los cuales quedan fuera del alcance de este informe.

2. Visión general del sistema

Propósito del sistema

El sistema tiene como propósito automatizar la gestión del pago de las cuotas de gastos comunes del edificio Nuevos Horizontes, permitiendo registrar pagos, calcular cuotas, gestionar la morosidad y generar informes de forma eficiente. Desde la perspectiva del negocio, su objetivo general es reducir la carga administrativa manual, disminuir la tasa de morosidad y mejorar el flujo de caja del edificio. Desde la perspectiva del usuario, busca ofrecer un acceso simple, transparente y en tiempo real al estado de sus gastos comunes, ya sea desde el rol de administrador o de residente representante.

Descripción general del sistema

El sistema es una aplicación web orientada a la gestión de pagos de gastos comunes, dirigida a dos tipos de usuarios: el administrador del edificio y el residente representante de cada departamento. Entre sus principales características se encuentran el registro y cálculo automático de cuotas, el pago en línea mediante una pasarela de pago externa, el envío de notificaciones automáticas por correo electrónico y la generación de informes financieros y de morosidad. Su público objetivo corresponde a administraciones de edificios de mediana escala que requieren digitalizar procesos actualmente realizados de forma manual.

Contexto del sistema

El sistema interactúa con dos actores principales: el **Administrador**, responsable de la gestión de departamentos, residentes, gastos comunes y generación de informes; y el

Residente representante, quien consulta y paga los gastos comunes de su departamento. Adicionalmente, el sistema se relaciona con dos servicios externos: una **pasarela de pago**, encargada de procesar las transacciones, y un **servicio de mailing**, encargado del envío de notificaciones y comprobantes por correo electrónico. Estas interacciones se encuentran representadas en el diagrama de contexto del sistema.

Funcionalidades claves

Entre las funcionalidades más relevantes del sistema se destacan:

- Registro y gestión de departamentos y residentes representantes.
- Registro de gastos comunes por parte del administrador, lo que desencadena automáticamente el cálculo de la cuota correspondiente a cada departamento.
- Pago en línea de cuotas a través de una pasarela de pago integrada.
- Cálculo automático de intereses por morosidad y segmentación de deudores por meses de atraso.
- Envío automático de notificaciones al residente representante ante la generación de una nueva cuota, la detección de morosidad, o la publicación de avisos administrativos generales.
- Generación de informes de flujo de caja, pagos, morosidad y gestión de departamentos.
- Autenticación y control de acceso diferenciado por rol.

Supuestos y dependencias

Se asume que todos los usuarios cuentan con acceso a internet y un dispositivo (computador o smartphone) con navegador web actualizado. El sistema depende de servicios externos en la nube, específicamente de una pasarela de pago para el procesamiento de transacciones y de un servicio de mailing para el envío de correos electrónicos, además de un servicio de autenticación (Cognito) y una base de datos NoSQL (DynamoDB) provistos por el proveedor de servicios cloud (AWS).

Restricciones o limitaciones iniciales

Dado que se trata de un sistema de escala reducida, con cálculos esporádicos (registro mensual de gastos comunes, pagos puntuales y consultas ocasionales), no se justifica mantener un servidor activo de forma permanente (24/7). Por este motivo, se optó por una arquitectura **Serverless** en AWS (API Gateway, Lambda, DynamoDB, entre otros), lo que permite simplificar la carga de administración de infraestructura, concentrar el desarrollo en la lógica de negocio y ser eficientes en el uso de recursos y costos en la nube, ya que solo se paga por la ejecución efectiva de las funciones. Asimismo, el sistema no contempla el almacenamiento de datos bancarios, delegando completamente esa responsabilidad a la pasarela de pago externa, por motivos de seguridad y cumplimiento normativo.

Objetivos de alto nivel

- Automatizar el cálculo y cobro de cuotas de gastos comunes, evitando errores de cálculo manual y fortaleciendo la integridad y precisión de los cobros, además de agilizar los tiempos administrativos para liberar recursos hacia otras tareas de mayor valor para la gestión del edificio.
- Reducir la tasa de morosidad mediante notificaciones automáticas y cálculo transparente de intereses.
- Entregar acceso transparente y en tiempo real a la información financiera del edificio.
- Garantizar la seguridad de la información mediante control de acceso por rol y buenas prácticas de manejo de datos sensibles.
- Sentar una base arquitectónica escalable que permita la futura incorporación de nuevas funcionalidades.

3. Requisitos arquitectónicos

Propósito de los requisitos arquitectónicos

Los requisitos arquitectónicos identificados a continuación corresponden a aquellos que tienen un impacto directo en las decisiones de diseño del sistema, estableciendo lo que la arquitectura debe cumplir para responder tanto a las necesidades funcionales del negocio como a las condiciones técnicas y de calidad del entorno en que operará el sistema.

Requisitos funcionales

A continuación se especifican individualmente los 30 requisitos funcionales del sistema, agrupados por área.

Gestión de residentes y departamentos

- **RF01:** El sistema debe permitir registrar los departamentos del edificio.
- **RF02:** El sistema debe permitir registrar un residente representante por cada departamento (propietario o arrendatario), responsable de la gestión de cobros.
- **RF03:** El sistema debe exigir que cada departamento tenga al menos un residente representante vigente para poder gestionar sus cobros y enviar sus notificaciones.
- **RF04:** El sistema debe exigir el registro de un correo electrónico del residente representante, que se utilizará como medio de contacto para notificaciones y como identificador de acceso (usuario) al sistema.
- **RF05:** El sistema debe permitir actualizar el residente representante de un departamento cuando este cambie (término de arriendo, cambio de arrendatario, venta).
- **RF06:** El sistema debe permitir dar de baja a un residente representante cuando deje de tener vínculo con el departamento, revocando su acceso. Si es el único representante del departamento, este queda en estado "sin representante": sus cuotas siguen generándose y quedan asociadas al departamento, el sistema alerta al administrador en el listado de departamentos y se suspende el envío de notificaciones hasta que se asigne un nuevo representante.

Gestión de pagos

- **RF07:** El sistema debe permitir registrar los gastos comunes del edificio (agua, gas, luz, mantención de ascensores, remuneraciones del personal, insumos, entre otros).
- **RF08:** El sistema debe calcular la cuota de gastos comunes de cada departamento, según las reglas de cálculo definidas por el negocio.
- **RF09:** El sistema debe permitir registrar los pagos realizados por el residente representante de cada departamento.
- **RF10:** El sistema debe mantener un historial de pagos por departamento.
- **RF11:** El sistema debe permitir consultar y filtrar los departamentos morosos, mostrando meses de atraso, monto adeudado por cuotas, intereses generados y el total adeudado de forma desglosada.
- **RF12:** El sistema debe enviar automáticamente una notificación por correo electrónico al residente representante de los departamentos morosos, en cada mes de atraso, independientemente de si ingresa a la aplicación.
- **RF13:** El sistema debe notificar automáticamente al residente representante, por correo electrónico y dentro de la aplicación, cuando se genere una nueva cuota disponible para pago.
- **RF14:** El interés se calcula al generarse cada nueva cuota, sobre el monto de cada cuota vencida, aplicando la tasa vigente en esa fecha. Si la tasa cambia, los intereses ya calculados no se recalculan; el nuevo valor rige solo para los cálculos posteriores.

Pago en línea y comprobantes

- **RF15:** El sistema debe permitir al residente representante pagar sus gastos comunes en línea, a través de una pasarela de pago integrada.
- **RF16:** El sistema debe registrar automáticamente el pago en el historial del departamento una vez confirmada la transacción por la pasarela.
- **RF17:** El sistema debe emitir un comprobante de pago tras una transacción exitosa y enviarlo automáticamente al correo electrónico registrado.
- **RF18:** El sistema debe permitir al residente representante descargar el comprobante de pago desde su historial en cualquier momento.

Consultas e informes

- **RF19:** El sistema debe generar informes de flujo de caja del edificio.
- **RF20:** El sistema debe generar informes detallados sobre pagos, morosidad y gestión de departamentos.

Acceso y funcionalidades del residente

- **RF21:** El sistema debe permitir al residente representante visualizar sus gastos comunes pendientes.
- **RF22:** El sistema debe permitir al residente representante visualizar su historial de gastos comunes pagados.
- **RF23:** El sistema debe permitir al residente representante visualizar las notificaciones asociadas a su cuenta dentro de la aplicación.
- **RF24:** El sistema debe implementar control de acceso diferenciado por rol: el administrador con permisos de gestión y el residente representante con acceso limitado a consulta y pago de su propia información.

Notificaciones administrativas

- **RF25:** El sistema debe permitir al administrador publicar notificaciones administrativas generales dirigidas a todos los residentes, enviándolas dentro de la aplicación y por correo electrónico.
- **RF26:** El sistema debe permitir a los residentes representantes visualizar las notificaciones administrativas generales dentro de la aplicación.

Credenciales y sesión

- **RF27:** El sistema debe generar una clave aleatoria para el primer ingreso de cada residente representante y enviarla a su correo electrónico registrado.
- **RF28:** El sistema debe exigir al residente representante el cambio de su clave en el primer inicio de sesión.
- **RF29:** El sistema debe exigir la rotación de contraseñas cada 6 meses a los usuarios con rol administrativo.
- **RF30:** El sistema debe permitir a cualquier usuario autenticado cerrar sesión de forma manual.

Requisitos no funcionales

Los 10 requisitos no funcionales se especifican a continuación.

Seguridad

- **RNF01:** El sistema debe proteger de forma segura las credenciales y los datos sensibles de los usuarios, delegando la gestión de contraseñas en Amazon Cognito, y almacenar de forma segura los registros de pago.
- **RNF02:** El sistema debe restringir el acceso a la información según el rol del usuario (administrador o residente representante).
- **RNF03:** El sistema debe generar registros de auditoría (logs) de las acciones críticas realizadas por los usuarios.
- **RNF04:** El sistema debe garantizar la recuperación de la información almacenada ante pérdida o corrupción de datos, apoyándose en las capacidades nativas de respaldo de DynamoDB (Point-in-Time Recovery), sin requerir un proceso de backup manual adicional.
- **RNF05:** El sistema no debe almacenar datos bancarios ni de tarjetas; toda la información de pago debe ser gestionada exclusivamente por la pasarela de pago externa.

Usabilidad

- **RNF06:** El sistema debe contar con una interfaz intuitiva que permita a residentes sin conocimientos técnicos consultar sus gastos y realizar pagos sin dificultad.
- **RNF07:** El sistema debe estar disponible en formato responsivo, adaptable a dispositivos móviles.

Rendimiento

- **RNF08:** El sistema debe actualizar el estado de la cuota y de la morosidad del departamento inmediatamente después de recibir la confirmación de la pasarela de pago, sin procesos por lotes, de modo que la siguiente consulta ya refleje el estado actualizado.
- **RNF09:** El sistema debe registrar el pago y mostrar la confirmación al usuario en un tiempo máximo de 5 segundos desde que recibe la confirmación de la pasarela de pago. El tiempo de procesamiento interno de la pasarela externa queda excluido de este plazo, por no depender del sistema.

Disponibilidad

- RNF10: El sistema debe tener una disponibilidad mensual del 99% (equivalente a un máximo de aproximadamente 7 horas de interrupción al mes), medida sobre los componentes administrados en AWS (frontend, API Gateway, Lambda, DynamoDB y Cognito). Se excluyen de esta métrica la pasarela de pago externa y las mantenciones programadas informadas con anticipación.

Restricciones tecnológicas o de contexto

El sistema debe implementarse bajo una arquitectura **serverless** en el proveedor de servicios cloud AWS, dado el bajo volumen de operaciones y la naturaleza esporádica de los cálculos (registro mensual de gastos comunes, pagos puntuales), lo que hace innecesario mantener un servidor activo de forma permanente. Esta decisión condiciona el uso de servicios como API Gateway, Lambda, DynamoDB, Cognito, S3, CloudFront y SES, EventBridge y determina que la lógica de negocio se implemente como funciones independientes en lugar de un servidor monolítico tradicional.

Esta restricción también se ve reforzada por el contexto organizacional del cliente: la administración del edificio no cuenta con un equipo de TI dedicado para la mantención del software, lo que descarta el uso de infraestructura tipo IaaS (como EC2), que requeriría tareas de aplicación de parches, actualización de sistema operativo y administración continua de servidores. El uso de funciones Lambda independientes por módulo funcional favorece además la mantenibilidad de la lógica de negocio en el tiempo, permitiendo ajustar o corregir una función específica sin necesidad de intervenir el resto del sistema.

Adicionalmente, el sistema no debe almacenar información bancaria de los residentes, por restricciones de seguridad y cumplimiento normativo, delegando dicha responsabilidad a la pasarela de pago externa.

4. Vista de escenarios (+1)

Propósito de la vista de escenarios

La vista de escenarios representa el sistema desde la perspectiva de los actores que interactúan con él, mostrando las funcionalidades que estos pueden ejecutar. Esta vista constituye el punto de partida del diseño arquitectónico, ya que permite validar que las

demás vistas (lógica, procesos, desarrollo y física) respondan de manera coherente a las necesidades reales de los usuarios del sistema.

Descripción general de los escenarios del sistema

Los escenarios del sistema se organizan en torno a dos actores principales: el administrador del edificio y el residente representante de cada departamento. El administrador interactúa con el sistema para gestionar residentes, departamentos, gastos comunes, morosidad, notificaciones e informes. El residente representante, por su parte, interactúa con el sistema para consultar sus gastos comunes, realizar pagos en línea, visualizar notificaciones y descargar comprobantes de pago. Adicionalmente, el sistema interactúa con dos actores secundarios de tipo sistema externo: la pasarela de pago, que procesa las transacciones, y el servicio de mailing, que envía notificaciones y comprobantes por correo electrónico.

Identificación de actores involucrados

Actor	Tipo	Rol / nivel de interacción
Administrador	Primario (usuario)	Gestiona departamentos, residentes, gastos comunes, morosidad, tasa de interés, notificaciones administrativas e informes. Requiere autenticación.
Residente representante	Primario (usuario)	Consulta sus gastos comunes, realiza pagos en línea, visualiza notificaciones y descarga comprobantes. Requiere autenticación.
Pasarela de pago	Secundario (sistema externo)	Procesa las transacciones de pago solicitadas por el sistema. No requiere autenticación de usuario final.
Servicio de mailing	Secundario (sistema externo)	Envía notificaciones y comprobantes por correo electrónico a solicitud del sistema.

Diagramas de la vista de escenarios

Diagrama de casos de uso general (nivel 1):

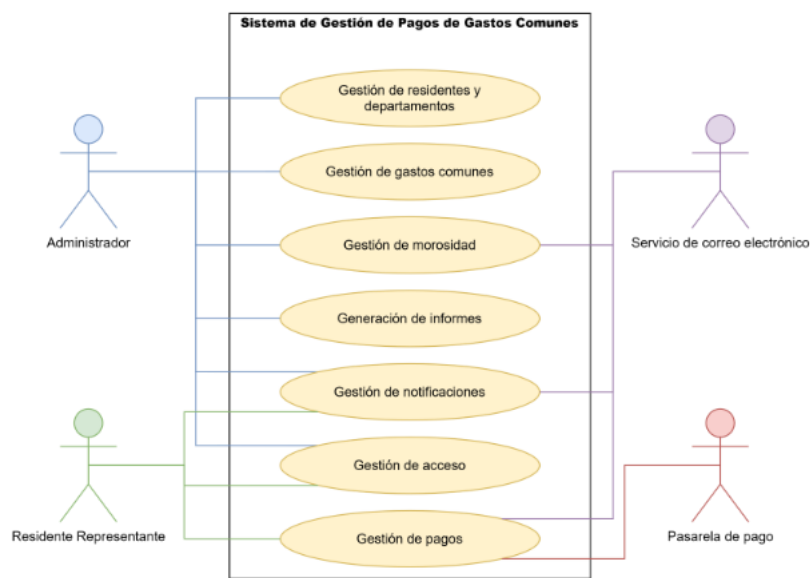
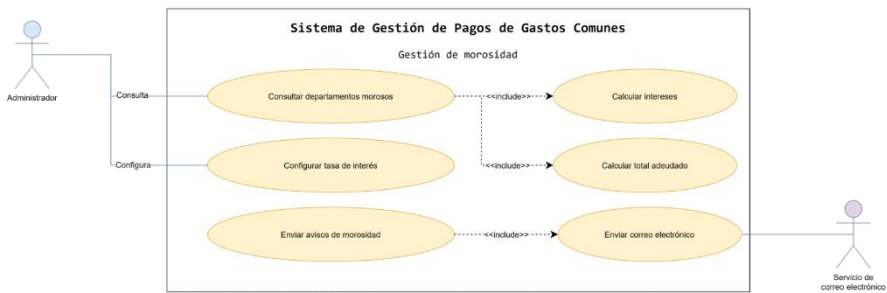
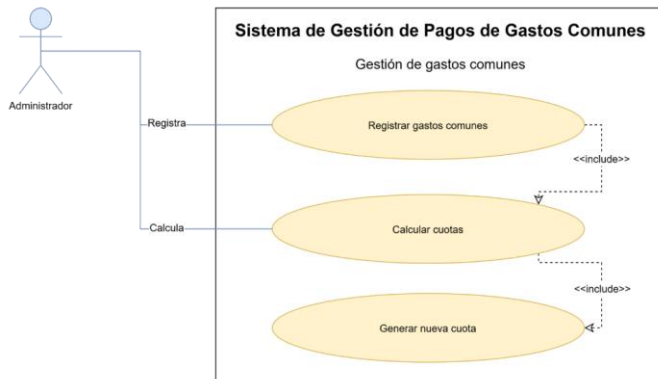
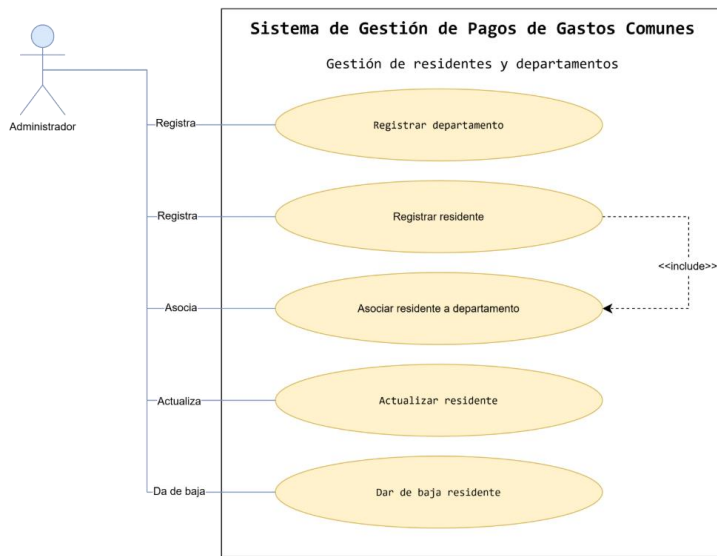
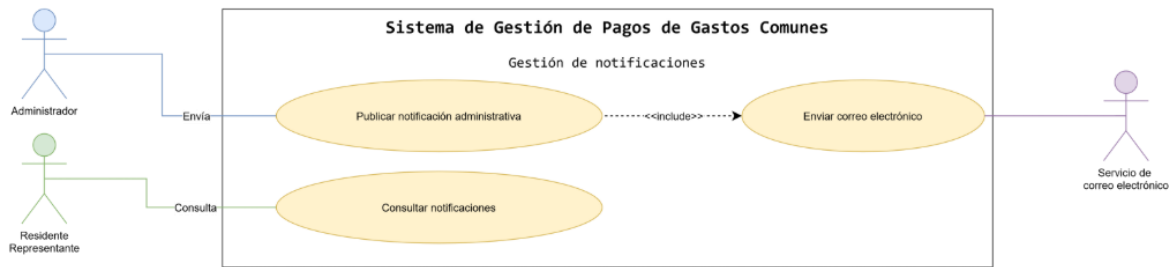
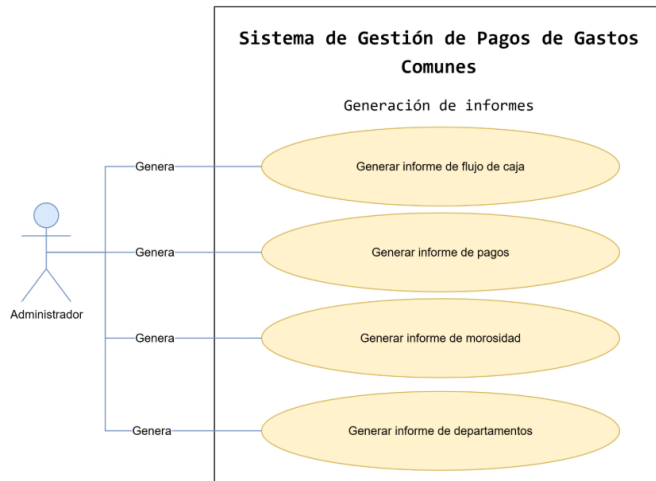


Diagrama de casos de uso específicos (nivel 2):







Descripción detallada de los escenarios o casos de uso

A continuación se presentan los casos de uso más representativos del sistema, seleccionados por su relevancia para el negocio y su nivel de complejidad, siguiendo la plantilla de especificación de casos de uso (nombre, actores involucrados, precondiciones, flujo principal, flujo alternativo y postcondición). El detalle completo de los 35 casos de uso especificados se encuentra disponible en el **Anexo A** de este documento.

CU10: Realizar pago

Actor principal: Residente Representante. Actor secundario: Pasarela de pago. Permite al residente representante realizar el pago en línea de una cuota de gastos comunes

pendiente, requiriendo sesión iniciada y al menos una cuota pendiente. El flujo principal contempla la consulta de gastos pendientes, selección de la cuota, confirmación del pago y envío de la solicitud a la pasarela de pago. Como flujo alternativo, si el residente cancela antes de confirmar, el sistema finaliza el proceso sin modificar el estado de la cuota.

CU08: Generar nueva cuota

Actor asociado: Administrador. Ejecución automática por el sistema tras el cálculo de cuotas. Registra la nueva cuota de gastos comunes calculada para cada departamento, dejándola disponible para su consulta y pago. Si el departamento presenta una cuota anterior impaga, este caso de uso extiende hacia CU18 (Calcular intereses), aplicando la tasa vigente sobre la deuda acumulada.

CU17: Consultar departamentos morosos

Actor principal: Administrador. Permite consultar los departamentos con cuotas vencidas, mostrando la cantidad de meses de atraso, el monto adeudado y los intereses previamente calculados. Incluye a CU19 (Calcular total adeudado) para presentar el detalle desglosado de la deuda.

CU22: Publicar notificación administrativa

Actor principal: Administrador. Permite publicar avisos generales dirigidos a todos los residentes (cambios de política, cortes de servicios, reparaciones), los que se registran, publican dentro de la aplicación y se envían por correo electrónico mediante CU23 (Enviar correo electrónico).

CU29: Iniciar sesión

Actores principales: Administrador y Residente Representante. Permite a los usuarios autorizados acceder al sistema mediante correo electrónico y contraseña. El sistema valida las credenciales, identifica el rol del usuario y concede acceso a las funcionalidades correspondientes. Si las credenciales no son válidas, el sistema rechaza el acceso e informa al usuario.

5. Vista lógica

Propósito de la vista lógica

La vista lógica representa la estructura funcional del sistema, mostrando las principales entidades del dominio del negocio, sus atributos, comportamientos y las relaciones que existen entre ellas. Su propósito es reflejar cómo el sistema organiza internamente la información relacionada con la gestión de residentes, departamentos, gastos comunes, pagos y notificaciones, sirviendo como base conceptual para su posterior implementación, independientemente de la tecnología de persistencia específica utilizada.

Descripción de la arquitectura funcional

El sistema adopta un diseño **orientado a objetos**, organizado por dominios de negocio (residentes y departamentos, pagos, gastos comunes, morosidad y notificaciones), en coherencia con la arquitectura serverless de la vista de desarrollo. La vista lógica se centra en las entidades del negocio y sus relaciones, sin considerar su distribución física ni la tecnología de persistencia.

Su alcance cubre las entidades necesarias para soportar los procesos principales del sistema: gestión de residentes y departamentos, registro de gastos comunes y cálculo de cuotas, procesamiento de pagos, cálculo de intereses por morosidad y envío de notificaciones.

Componentes funcionales

A continuación se describen las principales entidades identificadas en la vista lógica, junto con su responsabilidad dentro del sistema:

- **Usuario:** representa la cuenta de acceso del sistema (correo, rol, estado y fecha del último cambio de clave). Las contraseñas no se almacenan en el sistema: las gestiona Amazon Cognito, que también valida las credenciales y emite las sesiones
- **Administrador:** representa al usuario responsable de la gestión del sistema. Registra departamentos y residentes, gestiona gastos comunes, configura la tasa de interés y genera informes (RF01, RF07, RF14, RF19-RF20).

- **Residente:** representa al residente representante de un departamento. Consulta sus gastos pendientes e historial, realiza pagos y visualiza notificaciones (RF02-RF06, RF21-RF23).
- **Departamento:** representa una unidad del edificio, asociada a un residente representante vigente (RF01-RF03).
- **GastoComun:** representa un gasto registrado por el administrador (agua, luz, gas, etc.), utilizado como base para el cálculo de la cuota (RF07).
- **Cuota:** representa el monto que debe pagar un departamento en un período determinado, incluyendo el monto del gasto y, si corresponde, los intereses por morosidad (RF08, RF11, RF13-RF14).
- **TasaInteres:** representa la tasa vigente utilizada para calcular los intereses de las cuotas morosas, configurable por el administrador (RF14).
- **Pago:** representa una transacción de pago realizada por el residente representante, asociada a una cuota (RF09, RF15-RF16).
- **Comprobante:** representa el documento generado tras un pago exitoso, disponible para descarga o envío por correo (RF17-RF18).
- **Notificacion:** representa un aviso enviado al residente representante (nueva cuota, morosidad o aviso administrativo) (RF12-RF13, RF24-RF25).
- **PasarelaPago:** servicio externo responsable de procesar las transacciones de pago (RF15).
- **ServicioCorreoElectronico:** servicio externo responsable del envío de correos electrónicos asociados a comprobantes, notificaciones y credenciales (RF17, RF25, RF27).
- **Informe:** representa un informe generado por el administrador (flujo de caja, pagos, morosidad o departamentos), con su tipo, período consultado y fecha de generación (RF19-RF20).

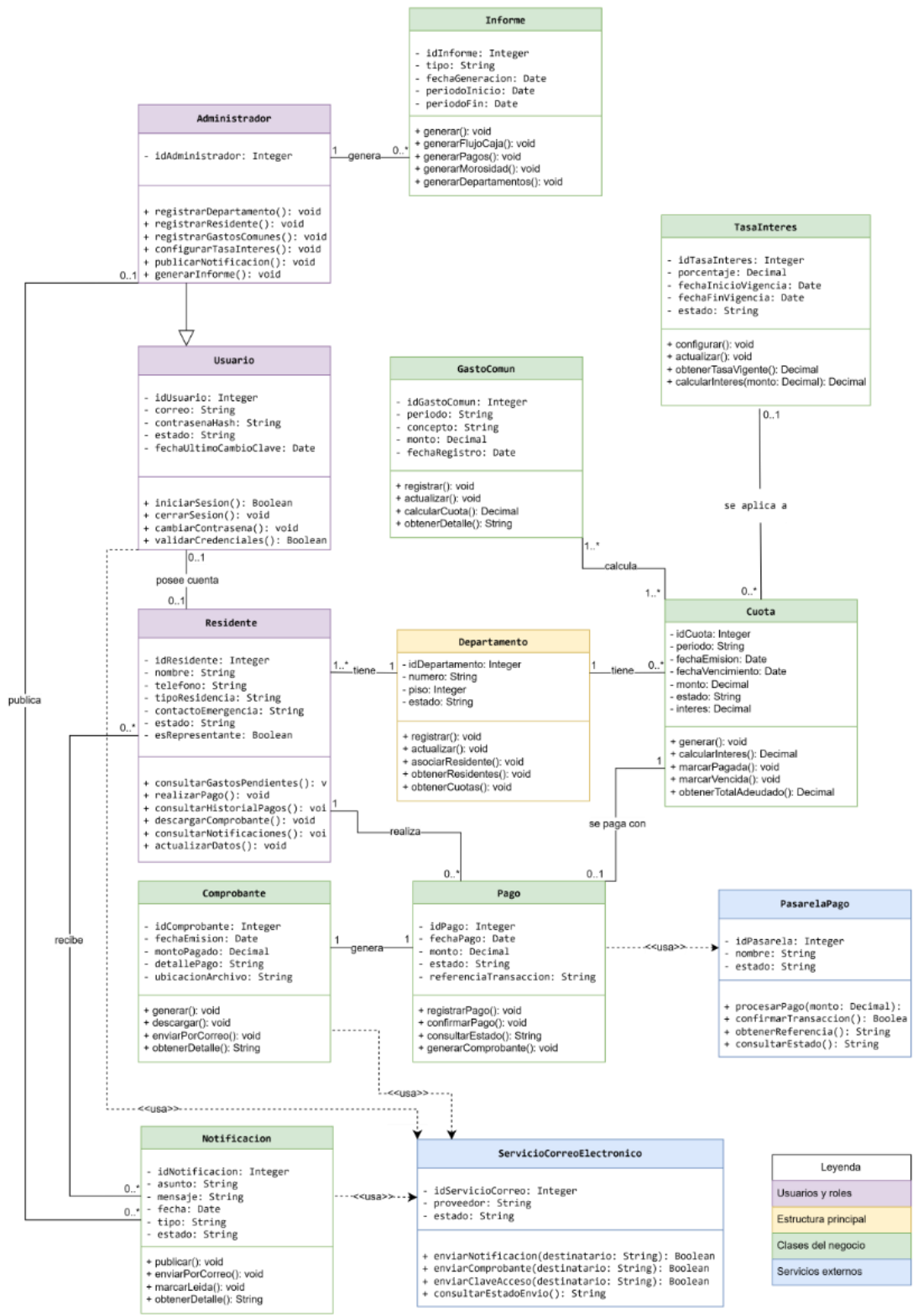
Relaciones entre componentes

Las principales relaciones identificadas entre las entidades del sistema son las siguientes:

- **Usuario — Residente / Administrador (1:1):** cada residente representante y cada administrador posee una cuenta de usuario asociada, mediante la cual se gestiona su autenticación.

- **Departamento — Residente (1:0..1):** un departamento tiene, como máximo, un residente representante vigente responsable de la gestión de cobros.
- **Departamento — Cuota (1:*):** un departamento genera múltiples cuotas a lo largo del tiempo, cada una con estado pendiente, vencida o pagada.
- **GastoComun — Cuota (asociación):** los gastos comunes registrados por el administrador son la base para el cálculo de las cuotas de cada departamento.
- **TasaInteres — Cuota (asociación):** la tasa de interés vigente al momento de generarse una nueva cuota se aplica al cálculo de intereses de las cuotas vencidas del mismo departamento.
- **Cuota — Pago (1:0..1):** una cuota se paga en su totalidad mediante un único pago; el sistema no contempla pagos parciales ni abonos. Una cuota pasa a estado pagada una vez registrado ese pago.
- **Pago — Comprobante (1:1):** todo pago exitoso genera un comprobante asociado.
- **Pago — PasarelaPago (dependencia, <<usa>>):** el registro de un pago depende de la confirmación entregada por la pasarela de pago externa.
- **Cuota — Notificacion (1:*):** una cuota puede generar distintas notificaciones a lo largo de su ciclo de vida (nueva cuota disponible, aviso de vencimiento).
- **Departamento — Notificacion (1:*):** un departamento en estado moroso (con al menos una cuota vencida) genera notificaciones de morosidad hacia su residente representante.
- **Comprobante / Notificacion — ServicioCorreoElectronico (dependencia, <<usa>>):** tanto el envío de comprobantes como el de notificaciones dependen del servicio externo de correo electrónico.

Diagramas de la vista lógica



6. Vista de procesos (vista dinámica)

Propósito de la vista de procesos

La vista de procesos describe el comportamiento dinámico del sistema durante su ejecución, mostrando cómo interactúan los usuarios, los servicios internos y los servicios externos para completar las operaciones principales. En el sistema de gestión de pagos de gastos comunes, esta vista permite representar especialmente el flujo asociado al pago de una cuota, desde la autenticación del residente representante y la consulta de sus gastos pendientes hasta el procesamiento de la transacción, actualización de la cuota, generación del comprobante y envío de la notificación correspondiente.

Esta vista permite analizar el orden de las actividades, los puntos de decisión y las respuestas del sistema ante situaciones alternativas, como credenciales inválidas, cancelación del pago, rechazo de la transacción por parte de la pasarela o errores durante el registro. De esta manera, se busca asegurar que los procesos mantengan una ejecución coherente y que el estado de las cuotas y pagos permanezca consistente durante todo el flujo.

El diagrama de actividades elaborado en la Semana 4 representa precisamente este proceso: inicio de sesión, consulta de gastos pendientes, selección de la cuota, confirmación, interacción con la pasarela de pago, registro del pago, actualización de la cuota, generación del comprobante y envío del correo electrónico.

Identificación de procesos y subprocessos activos

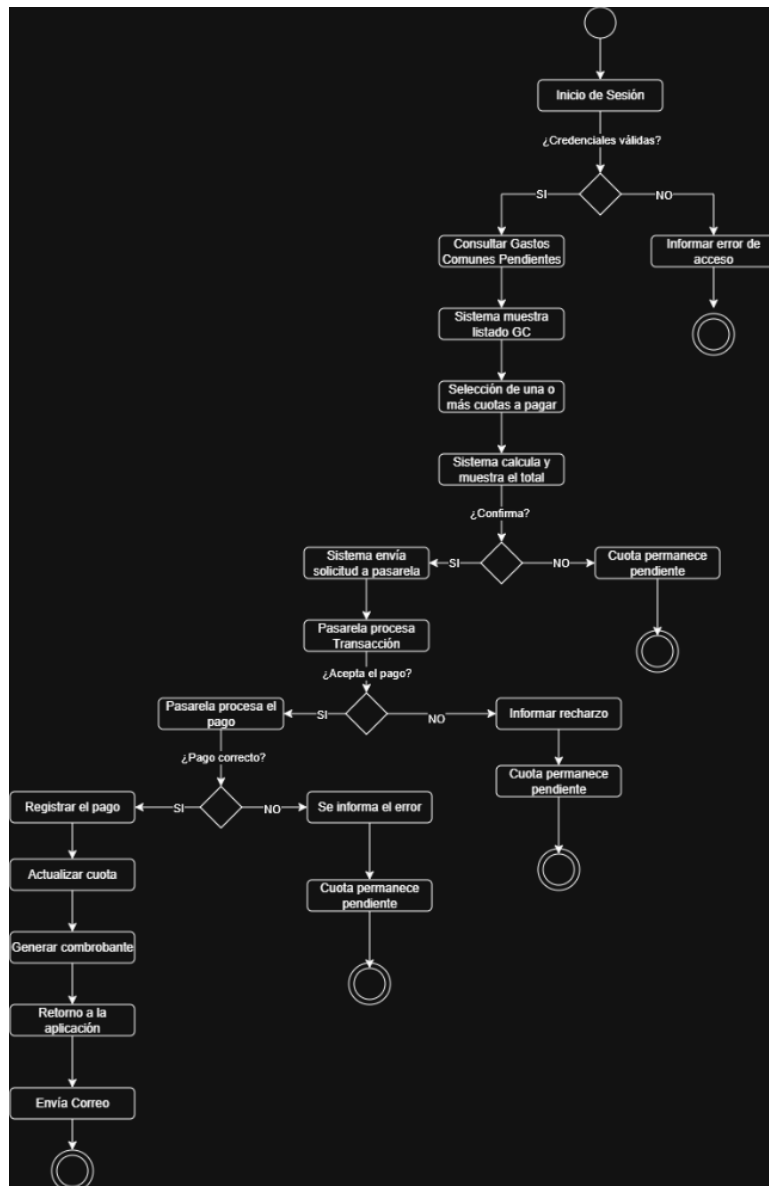
Los procesos que no inicia un usuario se ejecutan de dos formas. **Por evento:** cuando el administrador registra los gastos comunes del período, el sistema calcula y genera la cuota de cada departamento; en ese mismo flujo calcula los intereses de las cuotas anteriores impagas y notifica la nueva cuota al residente representante, por correo y dentro de la aplicación. **Por calendario:** una regla programada (Amazon EventBridge Scheduler) invoca mensualmente la función de morosidad, que marca como vencidas las cuotas cuya fecha de vencimiento ya pasó, actualiza los meses de atraso de cada departamento y envía el aviso de morosidad a su representante. Ninguno de los dos mecanismos requiere intervención del administrador.

Los principales procesos y subprocesos identificados durante la ejecución del sistema son los siguientes:

- **Autenticación de usuarios:** proceso que valida las credenciales ingresadas por el administrador o residente representante y determina el acceso a las funcionalidades correspondientes según el rol del usuario.
- **Consulta de gastos comunes pendientes:** proceso mediante el cual el residente representante consulta las cuotas pendientes asociadas a su departamento.
- **Realización de pago:** proceso iniciado por el residente representante al seleccionar una cuota pendiente y confirmar la operación de pago.
- **Procesamiento del pago:** subproceso que permite enviar la solicitud a la pasarela de pago externa y recibir la confirmación o rechazo de la transacción.
- **Registro del pago:** subproceso ejecutado después de una transacción exitosa, encargado de almacenar el pago en el historial del departamento.
- **Actualización de la cuota:** subproceso que modifica el estado de la cuota desde pendiente a pagada una vez confirmado y registrado el pago.
- **Generación de comprobante:** subproceso que crea el comprobante correspondiente a una transacción de pago realizada exitosamente y lo deja disponible para el residente representante.
- **Envío de correos electrónicos:** proceso utilizado para enviar comprobantes de pago, avisos de morosidad, nuevas cuotas y notificaciones administrativas mediante el servicio de correo electrónico.
- **Gestión de morosidad:** proceso encargado de identificar cuotas vencidas, calcular los intereses correspondientes según la tasa vigente y determinar el total adeudado por cada departamento moroso.
- **Generación de informes:** proceso iniciado por el administrador para obtener informes de flujo de caja, pagos, morosidad y gestión de departamentos.

Estos procesos se ejecutan principalmente en respuesta a acciones realizadas por los usuarios o a eventos generados por el propio sistema. La mayoría corresponde a procesos temporales, ya que comienzan ante una solicitud o evento específico y finalizan una vez completada la operación correspondiente.

Diagramas de la vista de procesos



7. Vista de desarrollo

Propósito de la vista de desarrollo

La vista de desarrollo tiene como propósito representar la organización interna del software desde la perspectiva del equipo de programación, mostrando cómo se estructura la solución en componentes, módulos y paquetes. Esta vista permite comprender la distribución de responsabilidades dentro del sistema, las dependencias entre sus partes y la forma en que se organiza el código para facilitar su implementación, mantenimiento y evolución.

En el sistema de gestión de pagos de gastos comunes, la solución se organiza en módulos asociados a las principales funcionalidades del negocio, tales como autenticación, gestión de residentes, gestión de pagos, gastos comunes, morosidad, generación de informes y envío de correos electrónicos. Esta separación permite mantener una alta cohesión dentro de cada módulo y reducir el acoplamiento entre funcionalidades.

El diagrama de componentes elaborado muestra componentes como **Autenticación**, **GestiónResidentes**, **GestiónPagos**, **GestiónGastosComunes**, **GestiónMorosidad**, **GestiónInformes**, **Mailing** y **PasarelaPago**, junto con las dependencias necesarias para ejecutar los procesos del sistema. Por su parte, el diagrama de paquetes organiza la solución en **Frontend**, **Data** y **Functions**, donde se agrupan los elementos relacionados con la interfaz, transferencia de datos y lógica funcional del sistema.

Esta organización facilita que los módulos puedan ser desarrollados, modificados y mantenidos de forma independiente, contribuyendo a una arquitectura modular y coherente con los requerimientos definidos para el sistema.

Descripción general de la organización del código

La organización del código se estructura de forma modular, separando las responsabilidades principales del sistema en paquetes claramente definidos. De acuerdo con el diagrama de paquetes elaborado, la solución se divide en tres bloques principales: **Frontend**, **Data** y **Functions**.

El paquete **Frontend** contiene los elementos relacionados con la interfaz de usuario y se organiza en los subpaquetes **pages**, **components** y **services**. El paquete **Data** contiene los objetos utilizados para transferir información entre las distintas partes del sistema,

representados mediante el subpaquete **dto**. Finalmente, el paquete **Functions** concentra la lógica funcional del sistema y se subdivide en los módulos **auth**, **residentes**, **pagos**, **gastosComunes**, **morosidad**, **informes**, **mailing** y **exceptions**.

Esta separación permite mantener las responsabilidades organizadas según su función, facilitando la comprensión de la estructura del proyecto, el mantenimiento del código y la modificación independiente de los distintos módulos del sistema. Además, la organización es coherente con el diagrama de componentes, donde las funcionalidades se encuentran separadas en componentes especializados como Autenticación, Gestión de Residentes, Gestión de Pagos, Gestión de Gastos Comunes, Gestión de Morosidad, Gestión de Informes y Mailing.

Herramientas, lenguajes y frameworks utilizados

Backend: Node.js con TypeScript: se optó por TypeScript sobre Node.js debido a sus bajos tiempos de inicio en frío (*cold start*), lo que refuerza la eficiencia buscada en la arquitectura serverless dado el uso esporádico del sistema. Además, cuenta con amplio soporte y documentación dentro del ecosistema de AWS.

Frontend: React (TypeScript): la interfaz de usuario se desarrollará utilizando **React** junto con TypeScript, manteniendo consistencia de lenguaje en todo el proyecto. Esta decisión permite compartir tipado y definiciones de datos (DTOs) entre el frontend y las funciones Lambda del backend, reduciendo errores de integración y facilitando la mantenibilidad del código. React, además, cuenta con amplio soporte comunitario y se integra de forma natural con el despliegue como aplicación de página única (SPA) sobre Amazon S3 y CloudFront.

Infraestructura y servicios en la nube (AWS): el sistema se apoya en un conjunto de servicios administrados de Amazon Web Services, seleccionados por su naturaleza serverless, que elimina la necesidad de administrar infraestructura y permite escalar de forma automática según la demanda:

- **Amazon API Gateway**, como punto de entrada para las peticiones HTTP hacia las funciones Lambda.
- **Amazon Cognito**, para la autenticación y gestión de credenciales de los usuarios.
- **Amazon DynamoDB**, como base de datos NoSQL para el almacenamiento de la información del sistema.

- **Amazon SES**, para el envío de notificaciones y comprobantes por correo electrónico.
- **Amazon S3 y Amazon CloudFront**, para el alojamiento y distribución del frontend.
- **Amazon Route 53**, para la resolución del dominio del sistema.
- **Amazon EventBridge Scheduler**, para la ejecución programada de procesos automáticos, como la actualización mensual del estado de morosidad de los departamentos.

Herramientas de modelado: el diseño de la arquitectura y los diagramas del sistema se elaboraron utilizando **Draw.io**, herramienta que permite construir diagramas UML (casos de uso, componentes, paquetes, actividad, despliegue y clases) de forma colaborativa y exportable, facilitando su documentación en el presente informe.

Diagramas de la vista de desarrollo

Diagrama de componentes.

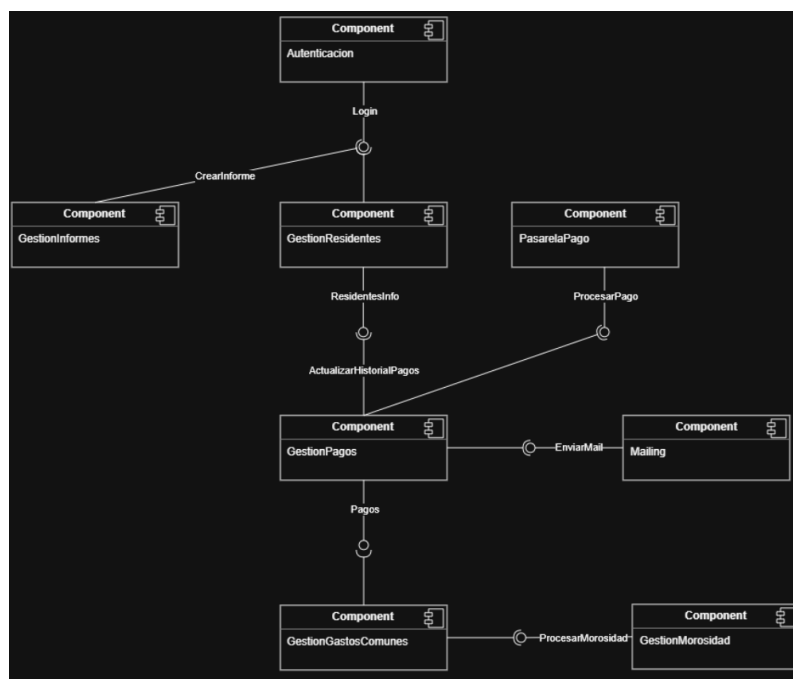
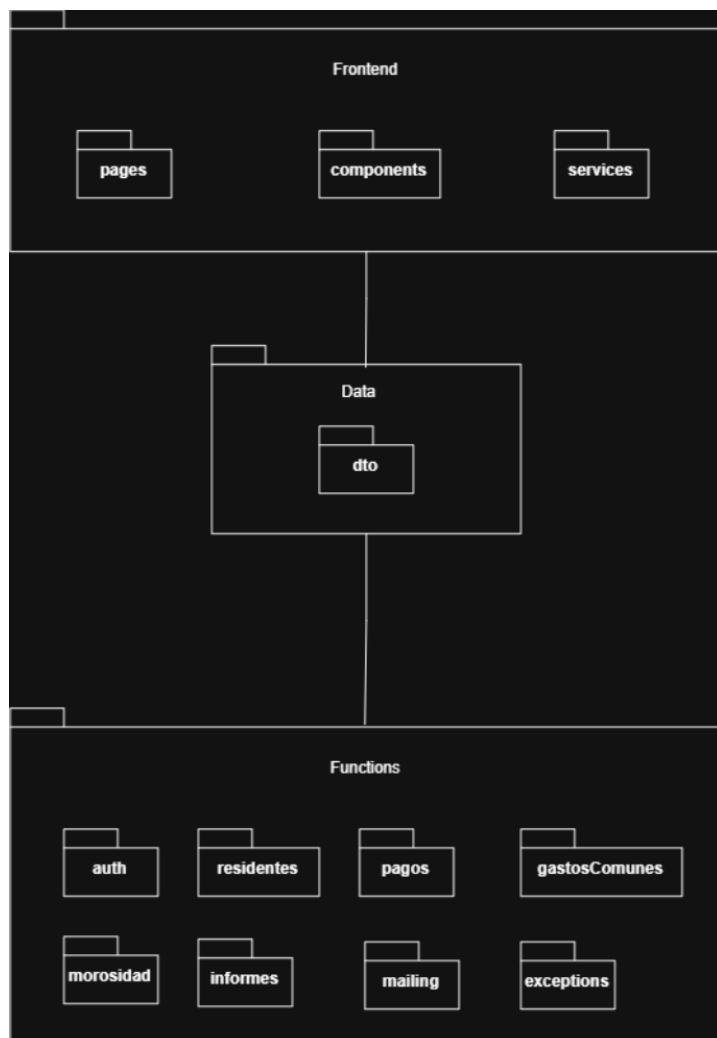


Diagrama de paquetes.



8. Vista física

Propósito de la vista física

La vista física tiene como propósito representar cómo se distribuyen los componentes del sistema sobre la infraestructura tecnológica donde será ejecutado. Esta vista permite identificar los nodos físicos o servicios de infraestructura, la ubicación de los componentes de software y las conexiones necesarias para que el sistema funcione correctamente.

En el sistema de gestión de pagos de gastos comunes, la vista física muestra la interacción de los usuarios con la aplicación web y la distribución de la solución sobre servicios en la

nube de AWS. El administrador y el residente representante acceden mediante un navegador web desde un computador o dispositivo móvil, mientras que la infraestructura considera servicios como Route 53, CloudFront, S3, Cognito, API Gateway, AWS Lambda, DynamoDB y AWS SES. Además, el sistema se comunica con una pasarela de pago externa para procesar las transacciones realizadas por los residentes.

Esta representación permite comprender dónde se ejecutan los distintos elementos del sistema y cómo se comunican entre sí, facilitando el análisis de aspectos relacionados con disponibilidad, seguridad, escalabilidad y conectividad de la solución.

Descripción general de la infraestructura

La infraestructura del sistema está basada en una arquitectura **serverless sobre AWS**, utilizando servicios administrados para soportar la aplicación web, la autenticación, la lógica de negocio, el almacenamiento de datos y el envío de correos electrónicos. Esta decisión responde al bajo volumen de operaciones y a la naturaleza esporádica de los procesos del sistema, evitando mantener servidores activos de forma permanente.

La solución considera los siguientes servicios de infraestructura: **Amazon Route 53** para la resolución de nombres de dominio, **Amazon CloudFront** para la distribución del contenido, **Amazon S3** para alojar el frontend de la aplicación, **Amazon Cognito** para la autenticación de usuarios, **Amazon API Gateway** para gestionar las solicitudes hacia el backend, **AWS Lambda** para ejecutar la lógica de negocio, **Amazon DynamoDB** para el almacenamiento de datos y **Amazon SES** para el envío de correos electrónicos.

Además, el sistema se integra con una **pasarela de pago externa**, responsable de procesar las transacciones de los residentes representantes, sin que el sistema almacene información bancaria o de tarjetas.

Identificación y caracterización de nodos

Los nodos identificados en la infraestructura del sistema son los siguientes:

- **Dispositivo del usuario:** corresponde al computador o dispositivo móvil utilizado por el administrador o residente representante para acceder a la aplicación mediante un navegador web.

- **Amazon Route 53:** servicio encargado de gestionar la resolución del nombre de dominio utilizado para acceder al sistema.
- **Amazon CloudFront:** servicio encargado de distribuir el contenido de la aplicación web hacia los usuarios.
- **Amazon S3:** servicio utilizado para alojar los archivos correspondientes al frontend de la aplicación.
- **Amazon Cognito:** servicio encargado de la autenticación y gestión del acceso de los usuarios.
- **Amazon API Gateway:** servicio que recibe las solicitudes realizadas desde la aplicación y las dirige hacia las funciones encargadas de ejecutar la lógica de negocio.
- **AWS Lambda:** entorno donde se ejecutan las funciones asociadas a los distintos módulos del sistema, como autenticación, residentes, pagos, gastos comunes, morosidad e informes.
- **Amazon DynamoDB:** servicio de base de datos utilizado para almacenar la información gestionada por el sistema.
- **Amazon SES:** servicio utilizado para el envío de correos electrónicos relacionados con notificaciones, avisos y comprobantes.
- **Pasarela de pago externa:** servicio externo encargado de procesar las transacciones de pago realizadas por los residentes representantes.

Asignación de componentes de software a nodos

La asignación de los componentes de software a los nodos de infraestructura se realiza de acuerdo con el diagrama de despliegue definido para el sistema:

- **Dispositivo PC o móvil:** contiene el navegador web utilizado por el administrador y el residente representante para acceder al sistema.
- **Amazon Route 53:** gestiona el acceso al dominio del sistema mediante DNS.
- **Amazon CloudFront:** distribuye el contenido de la aplicación hacia los usuarios.
- **Amazon S3:** aloja el frontend estático de la aplicación web.
- **Amazon Cognito:** contiene las funciones relacionadas con la autenticación y el control de acceso de los usuarios.
- **Amazon API Gateway:** actúa como punto de entrada para las solicitudes dirigidas a la lógica del sistema.

- **AWS Lambda:** ejecuta las funciones serverless correspondientes a la lógica de negocio de la aplicación.
- **Amazon DynamoDB:** almacena la información utilizada y generada por el sistema mediante una base de datos NoSQL.
- **Amazon SES:** soporta el componente de mailing encargado del envío de correos electrónicos.
- **Pasarela de pago:** corresponde a un servicio externo encargado del procesamiento de las transacciones de pago.

Conectividad entre nodos

La conectividad entre los nodos permite integrar el acceso de los usuarios con los distintos servicios que componen la solución. El administrador y el residente representante acceden al sistema desde un computador o dispositivo móvil mediante un navegador web.

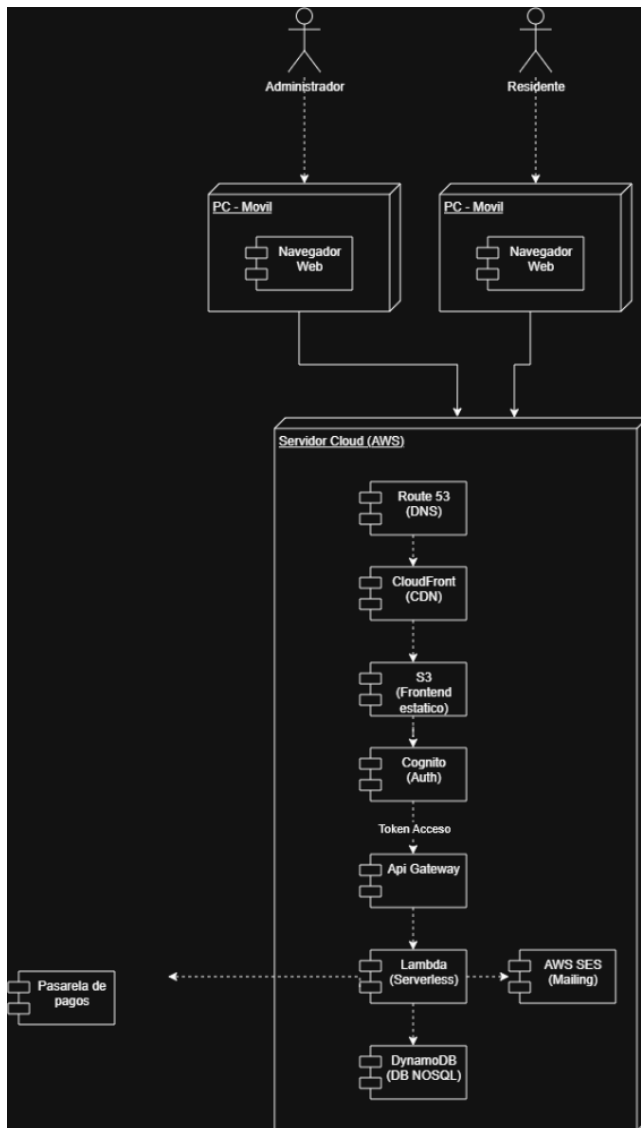
La comunicación entre los nodos se organiza de la siguiente manera:

- **Route 53** gestiona la resolución del nombre de dominio utilizado para acceder al sistema.
- **CloudFront** distribuye el contenido de la aplicación hacia los usuarios.
- **Amazon S3** aloja y entrega el frontend estático de la aplicación.
- **Amazon Cognito** gestiona la autenticación y el acceso de los usuarios.
- **API Gateway** recibe las solicitudes generadas desde la aplicación y las dirige hacia la lógica de negocio.
- **AWS Lambda** ejecuta las funciones asociadas a las operaciones del sistema.
- **DynamoDB** permite almacenar y recuperar la información utilizada por las funciones del sistema.
- **Amazon SES** gestiona el envío de correos electrónicos asociados a notificaciones y comprobantes.
- La **pasarela de pago externa** se comunica con la lógica del sistema para procesar las transacciones de pago.

Esta conectividad permite integrar el acceso de los usuarios, la autenticación, el procesamiento de solicitudes, el almacenamiento de datos, el envío de correos electrónicos y el procesamiento de pagos dentro de una misma solución tecnológica.

Diagrama de la vista física

Diagrama de despliegue.



9. Decisiones arquitectónicas

Propósito de la sección

Esta sección tiene como propósito documentar las principales decisiones arquitectónicas adoptadas para el sistema, considerando los requerimientos funcionales y no funcionales, las restricciones tecnológicas y las necesidades del negocio.

Las decisiones tomadas buscan definir una solución que permita automatizar la gestión de gastos comunes, facilitar el mantenimiento del sistema, reducir la administración de infraestructura y mantener una adecuada integración con servicios externos, como la pasarela de pago y el servicio de correo electrónico.

Además, esta sección permite dejar registro de las alternativas consideradas y de los criterios utilizados para seleccionar la arquitectura propuesta, facilitando la comprensión, mantenimiento y futura evolución de la solución.

Resumen de decisiones clave tomadas

Las principales decisiones arquitectónicas definidas para el sistema son las siguientes:

- **Adopción de una arquitectura Serverless en AWS:** se establece una arquitectura sin servidores debido al bajo volumen de operaciones y a la ejecución esporádica de procesos como el registro mensual de gastos comunes, pagos y consultas. Esto evita mantener servidores activos permanentemente.
- **Uso de AWS Lambda para la lógica de negocio:** las funcionalidades del sistema se ejecutan mediante funciones independientes, permitiendo separar los distintos módulos funcionales y ejecutar los recursos únicamente cuando sean requeridos.
- **Uso de Amazon DynamoDB como base de datos NoSQL:** la información del sistema se almacena mediante DynamoDB como servicio de persistencia administrado dentro de la infraestructura AWS.
- **Uso de Amazon Cognito para autenticación:** la gestión de credenciales y autenticación de los usuarios se delega a Cognito, apoyando el control de acceso diferenciado entre administrador y residente representante.
- **Integración con una pasarela de pago externa:** el procesamiento de los pagos se delega a un servicio externo. El sistema no almacena información bancaria ni datos de tarjetas de los residentes.
- **Uso de servicios administrados para la aplicación web:** la solución contempla Route 53, CloudFront y S3 para el acceso y distribución del frontend, además de API Gateway para recibir las solicitudes dirigidas hacia las funciones del sistema.
- **Uso de un servicio de correo electrónico:** el envío de notificaciones, avisos y comprobantes se realiza mediante un servicio de mailing, representado en la infraestructura mediante Amazon SES.

Estas decisiones son coherentes con las restricciones tecnológicas y de seguridad ya definidas para el sistema.

Alternativas consideradas

Durante la definición de la arquitectura se consideraron alternativas distintas a la solución Serverless finalmente adoptada.

- **Infraestructura IaaS mediante servidores virtuales, como Amazon EC2:** esta alternativa fue descartada debido a que requeriría administración continua de servidores, aplicación de parches, actualizaciones del sistema operativo y tareas de mantenimiento de infraestructura. Considerando que la administración del edificio no cuenta con un equipo de TI dedicado, esta opción aumentaría la carga operativa.
- **Servidor monolítico tradicional:** también se descartó mantener un servidor activo de forma permanente, debido al bajo volumen de operaciones y a la naturaleza esporádica de procesos como el registro mensual de gastos comunes, pagos y consultas. En este contexto, mantener infraestructura disponible permanentemente generaría un uso innecesario de recursos.

Frente a estas alternativas, se seleccionó una arquitectura Serverless basada en servicios administrados de AWS, permitiendo ejecutar la lógica de negocio únicamente cuando sea requerida y reducir las tareas asociadas a la administración de infraestructura.

Lecciones aprendidas (opcional)

Una de las principales lecciones de este proyecto fue comprender que crear entender las necesidades del cliente no es lo mismo que documentarlas de forma estructurada y transparente. Elaborar formalmente los requisitos, casos de uso y decisiones arquitectónicas permitió detectar ambigüedades y vacíos que, de no haberse abordado a tiempo, podrían haber derivado en una solución alejada de las necesidades reales del negocio. Este proceso reforzó la importancia de documentar antes de construir, ya que permite entregar valor en el menor tiempo posible, evitando incurrir en costos innecesarios asociados a soluciones arquitectónicamente deficientes o con código difícil de mantener.

10. Patrones y estilos arquitectónicos

Propósito de la sección

Esta sección tiene como propósito identificar y documentar los patrones y estilos arquitectónicos utilizados en la solución, explicando cómo estos permiten organizar los componentes del sistema y orientar su implementación.

La definición de estos patrones y estilos permite mantener una estructura coherente entre los distintos módulos de la aplicación, facilitando la comprensión de la arquitectura, su mantenimiento y su evolución.

Estilo arquitectónico general adoptado

El estilo arquitectónico general adoptado para el sistema es una **arquitectura Serverless en la nube**, implementada sobre servicios de AWS. Este estilo permite ejecutar la lógica de negocio mediante funciones que se activan cuando son requeridas, sin necesidad de mantener un servidor de aplicación activo de forma permanente.

La elección de este estilo responde al bajo volumen de operaciones y a la naturaleza esporádica de procesos como el registro mensual de gastos comunes, los pagos y las consultas. La solución utiliza servicios administrados como **API Gateway, AWS Lambda, DynamoDB, Cognito, S3, CloudFront y SES**, reduciendo las tareas asociadas a la administración y mantenimiento de infraestructura.

Además, este enfoque se ajusta al contexto del sistema, ya que permite mantener una infraestructura flexible y administrada por servicios en la nube, evitando la necesidad de operar servidores propios de manera permanente.

Patrones arquitectónicos específicos aplicados

Serverless (Function as a Service): es el patrón arquitectónico principal adoptado por el sistema. Bajo este patrón, la lógica de negocio se organiza como un conjunto de funciones independientes (AWS Lambda), cada una responsable de un dominio funcional específico (autenticación, gestión de residentes, pagos, morosidad, notificaciones, informes), que se ejecutan únicamente en respuesta a un evento (una petición HTTP a través de API Gateway) y sin necesidad de mantener un servidor activo de forma permanente. Este patrón se aplica en la totalidad de la lógica de backend del sistema, y es coherente con las restricciones

definidas previamente: bajo volumen de operaciones, ausencia de un equipo de TI dedicado a la mantención de servidores, y la necesidad de optimizar costos pagando solo por el tiempo de ejecución efectivo.

Broker (a través de API Gateway): el sistema utiliza a **Amazon API Gateway** como intermediario (*broker*) entre las peticiones del cliente (frontend) y las funciones Lambda que resuelven cada solicitud. API Gateway se encarga de recibir las peticiones HTTP, enrutarlas hacia la función Lambda correspondiente, y validar la autenticación del usuario mediante su integración con Amazon Cognito, desacoplando al cliente del conocimiento de la implementación interna de cada función.

Justificación de los patrones y estilos seleccionados

La elección del patrón Serverless responde directamente a las restricciones y necesidades identificadas en el contexto del negocio: el edificio Nuevos Horizontes no cuenta con un equipo de TI dedicado a la mantención de servidores, y tanto el volumen de operaciones como la cantidad de usuarios del sistema son reducidos, del orden de decenas de residentes y solicitudes puntuales, no miles de solicitudes concurrentes, lo que hace ineficiente mantener infraestructura activa de forma permanente para sostener una carga de trabajo tan baja. Este patrón permite que el equipo de desarrollo concentre sus esfuerzos en la lógica de negocio, delegando la administración de infraestructura, el escalamiento y la disponibilidad al proveedor cloud, y pagando únicamente por el tiempo de ejecución efectivo de las funciones, evitando así costos fijos asociados a un servidor sobredimensionado para la escala real del sistema.

La incorporación del patrón Broker, a través de Amazon API Gateway, se justifica por la necesidad de desacoplar al cliente (frontend) de la implementación interna de cada función Lambda, centralizando además la validación de autenticación mediante su integración con Amazon Cognito, sin que cada función deba implementar dicha lógica de forma independiente.

En conjunto, ambos patrones favorecen la **mantenibilidad** del sistema en el tiempo: al estar la lógica de negocio distribuida en funciones independientes por dominio, es posible corregir o ajustar un módulo específico sin necesidad de intervenir el resto del sistema, lo cual resulta especialmente relevante considerando que la administración del edificio no dispone

de personal técnico propio para sostener el mantenimiento de una arquitectura más tradicional.

Consideraciones o limitaciones

La arquitectura Serverless presenta algunas limitaciones a considerar. El **cold start** puede generar latencia adicional en la primera ejecución de una función tras un período de inactividad, aunque el uso de Node.js/TypeScript mitiga parcialmente este efecto. La distribución de la lógica en múltiples funciones independientes también dificulta la trazabilidad y depuración de errores en comparación con una aplicación monolítica. Adicionalmente, API Gateway introduce un punto adicional de latencia y podría constituir un punto único de falla si su configuración de autenticación falla.

11. Prototipos de interfaz

Esta sección documenta el prototipado de la interfaz del sistema, desarrollado en dos etapas: un prototipo de baja fidelidad, orientado a validar la estructura y las funcionalidades clave, y un prototipo de alta fidelidad, orientado a validar el estilo visual, la navegación y la experiencia de uso.

Prototipo

El prototipo representa la interfaz móvil del sistema de gestión de pagos de gastos comunes. Su propósito es validar, antes de construir el software, que la disposición de los elementos, los flujos de navegación y los mensajes del sistema respondan a las necesidades de sus dos usuarios: el Administrador y el Residente representante. Se optó por el formato móvil porque ambos perfiles gestionarán el sistema principalmente desde su celular, en coherencia con el requisito de usabilidad definido en la arquitectura.

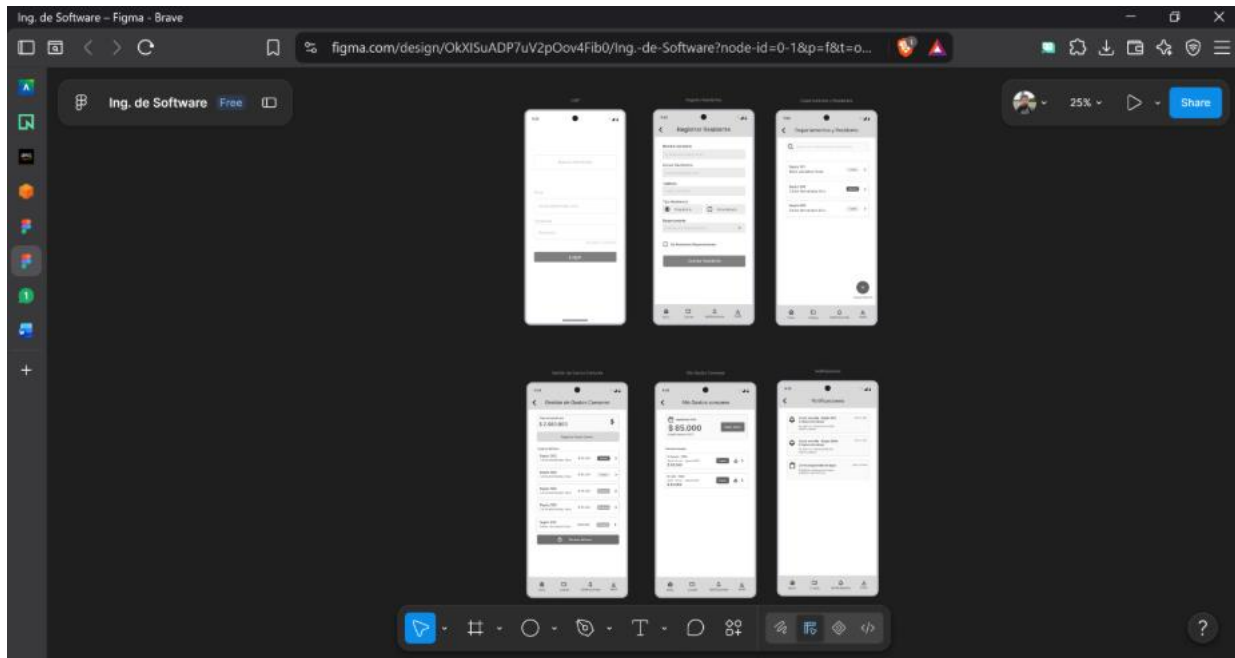
Prototipo de baja fidelidad

Para identificar las necesidades, motivaciones y frustraciones de los usuarios se aplicó la técnica de brainstorming estructurado. Se definió como pregunta guía qué debe incluir la interfaz para ser fácil de usar por un grupo diverso de personas con distintos niveles de manejo tecnológico. Las ideas generadas se agruparon en cuatro categorías: simplicidad visual, navegación intuitiva, legibilidad y accesibilidad, y retroalimentación clara.

A partir de estas categorías se diseñó un wireframe en escala de grises, sin identidad visual, compuesto por seis pantallas que cubren las cuatro funcionalidades clave:

- Inicio de sesión (compartida por ambos roles).
- Registro de residentes (Administrador).
- Listado de departamentos y residentes, con indicador de estado de morosidad (Administrador).
- Gestión de gastos comunes y cuotas del edificio (Administrador).

- Mis gastos comunes, con cuota pendiente e historial de pagos (Residente).
- Notificaciones y alertas por morosidad.



Login:

Login

9:30

Nuevos Horizontes

Email

usuario@ejemplo.com

Password

Password

[Recuperar Contraseña](#)

Login

Registro de residentes:

Registro Residentes

9:30

Registrار Residente

Nombre Completo

Ej: María González Pérez

Correo Electronico

usuario@ejemplo.com

Teléfono

+569 12345678

Tipo Residencia

☒ Propietario

☐ Arrendatario

Departamento

Seleccionar Departamento

☐ Es Residente Representante

Guardar Residente

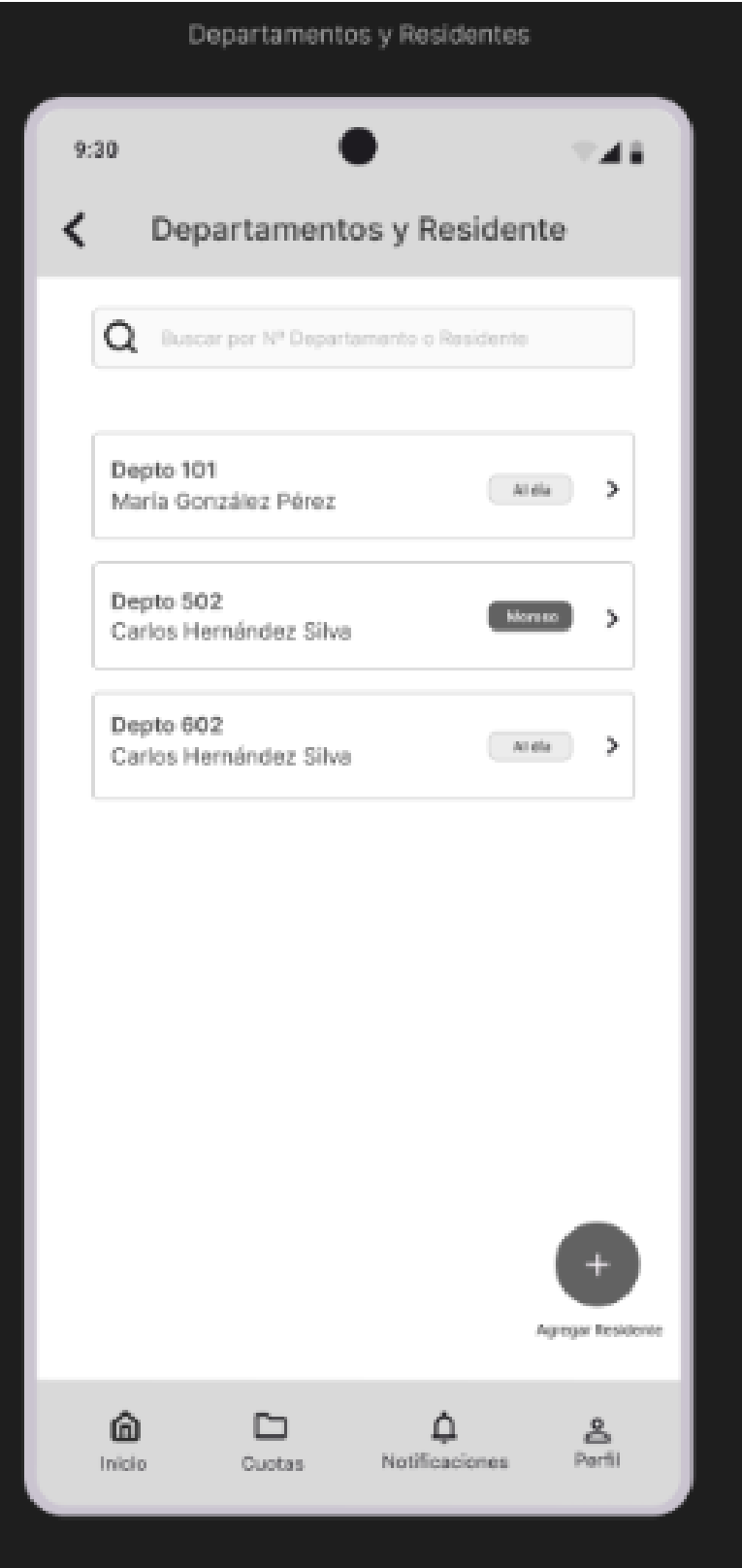
Inicio

Cuotas

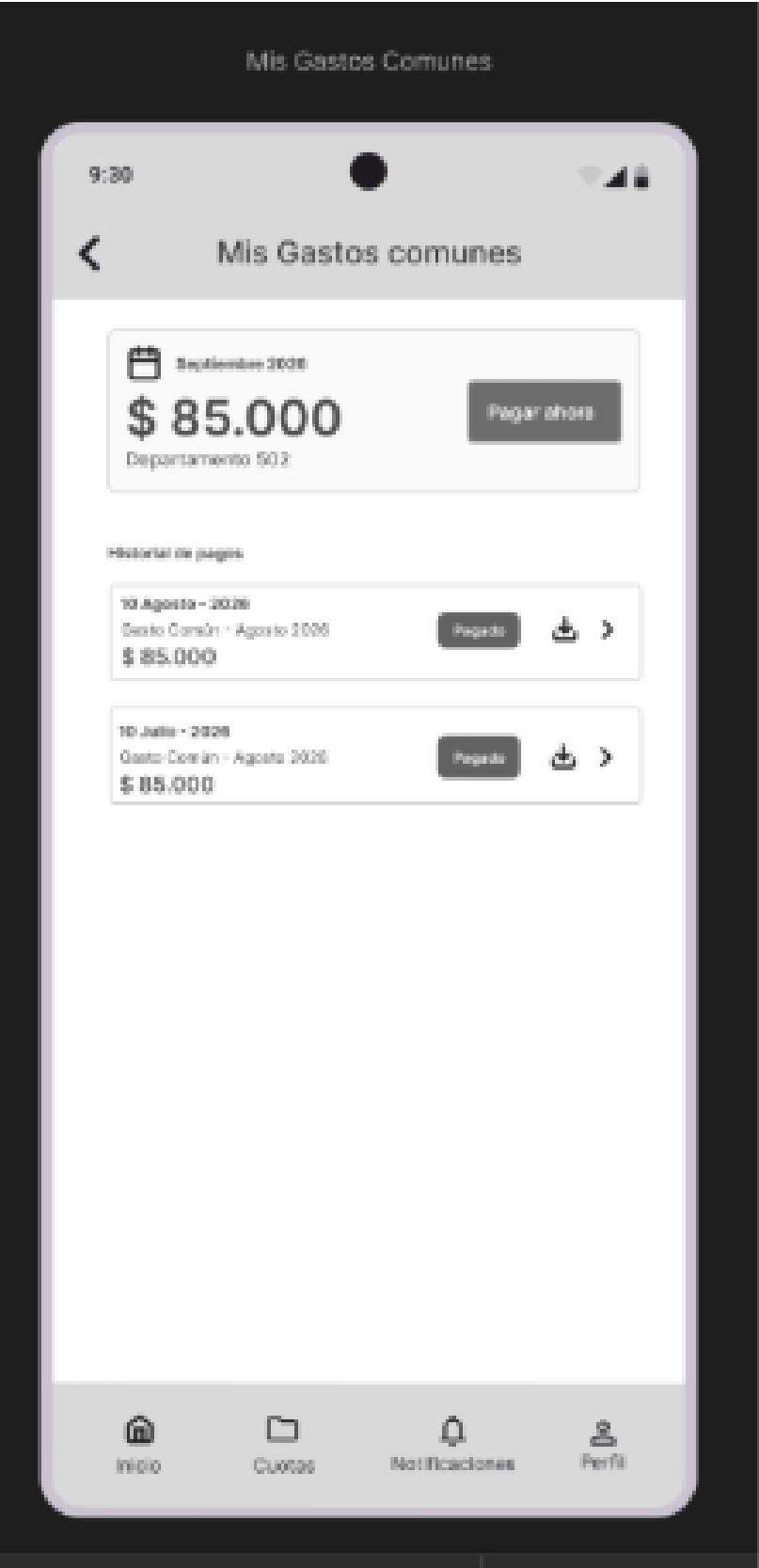
Notificaciones

Perfil

Departamentos y Residentes:



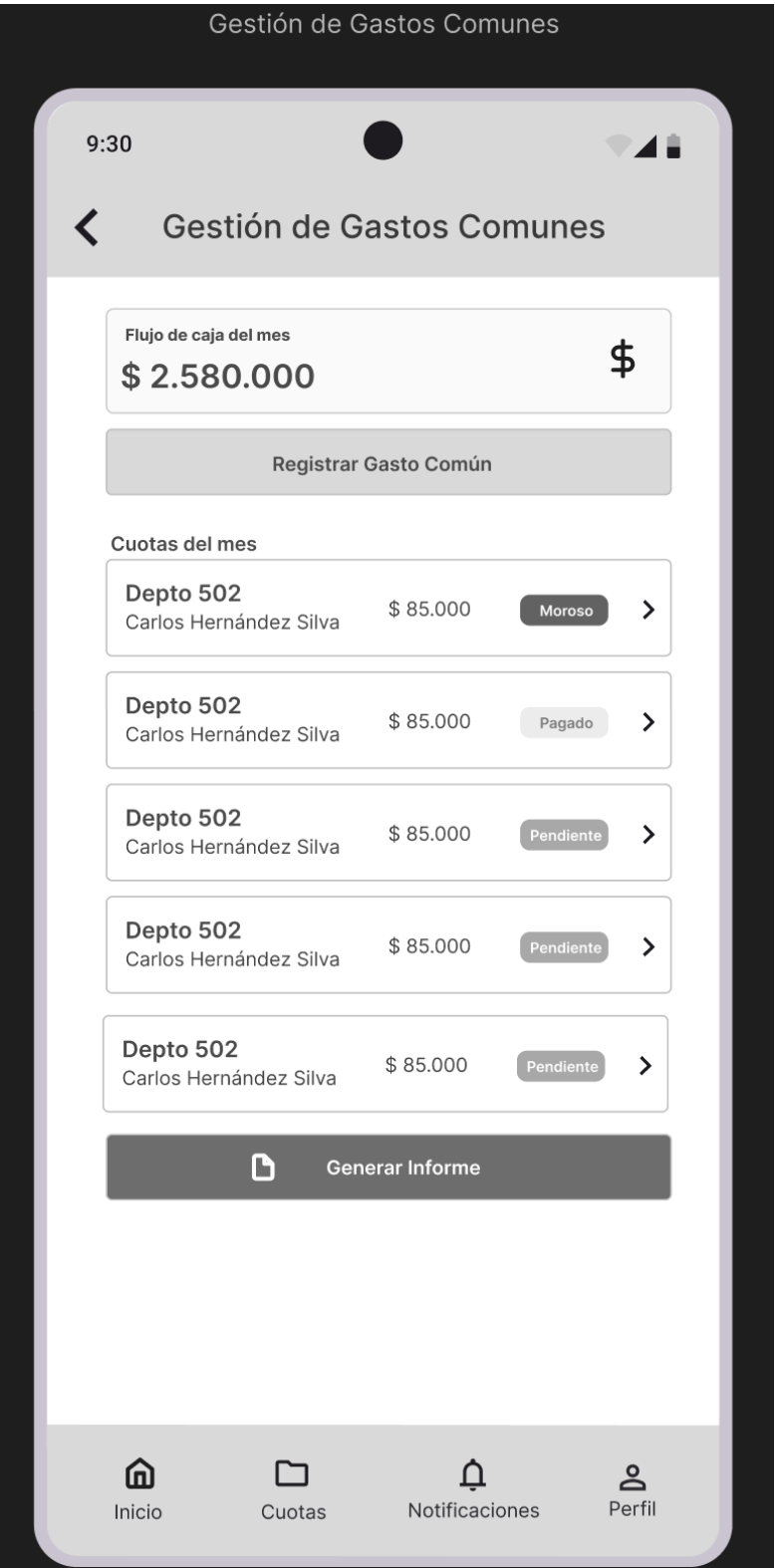
Gastos Comunes:



Notificaciones:



Gestión de Gastos Comunes:



Prototipo de alta fidelidad

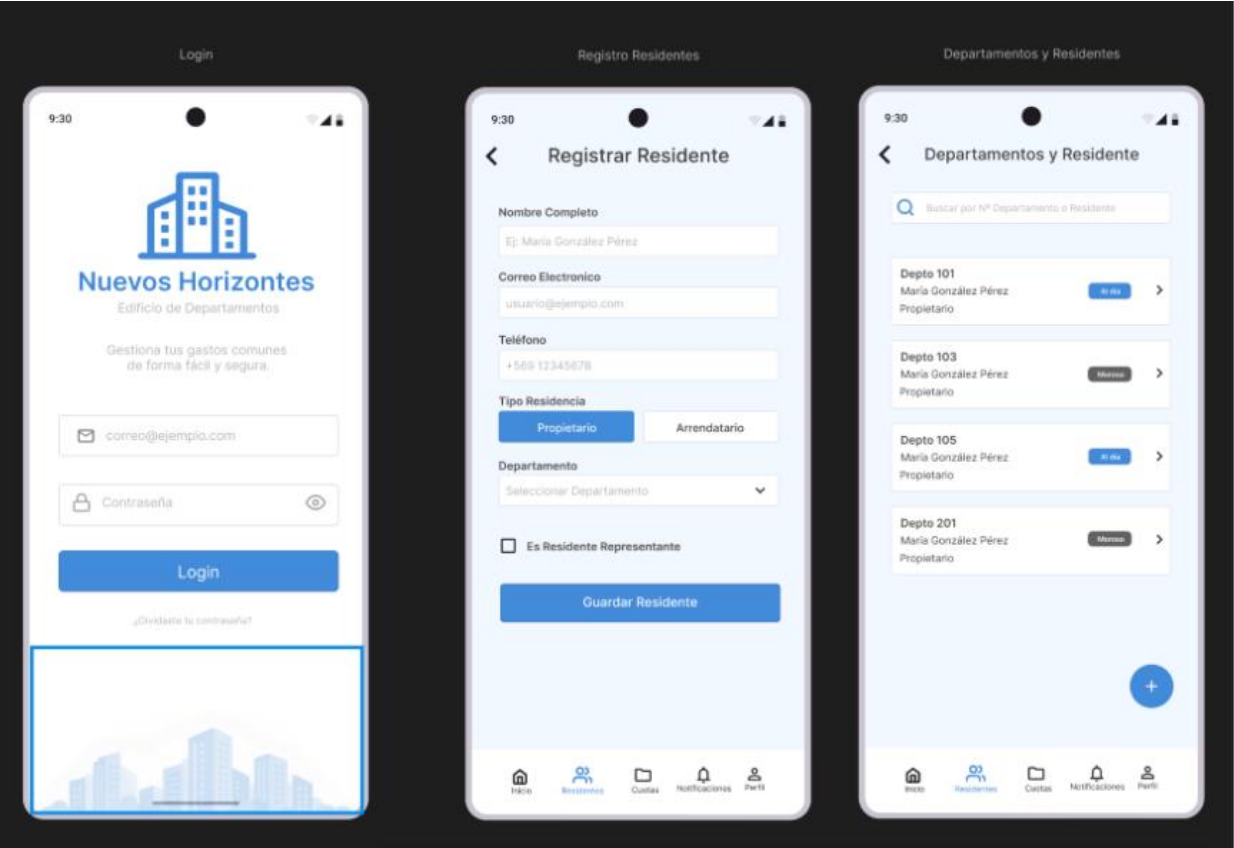
Sobre la base del wireframe se desarrolló el prototipo de alta fidelidad para ambos roles del sistema, aplicando un sistema visual completo: paleta con un color primario azul y neutros de apoyo, tipografía sans-serif con escala jerárquica de títulos y texto, íconos de estilo outline con etiqueta de texto, y componentes reutilizables como tarjetas, botones y badges de estado.

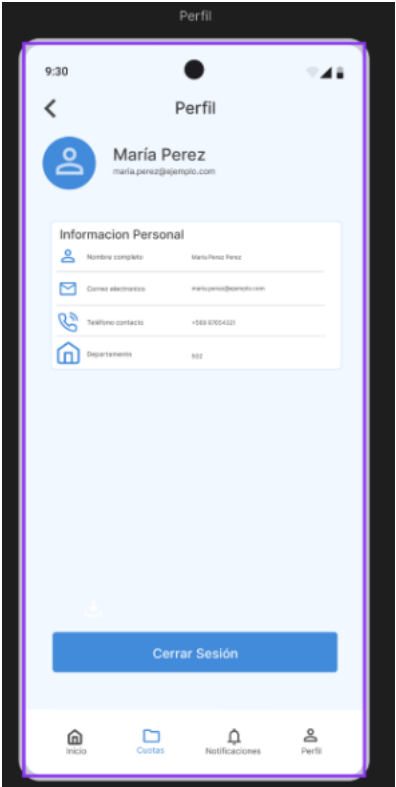
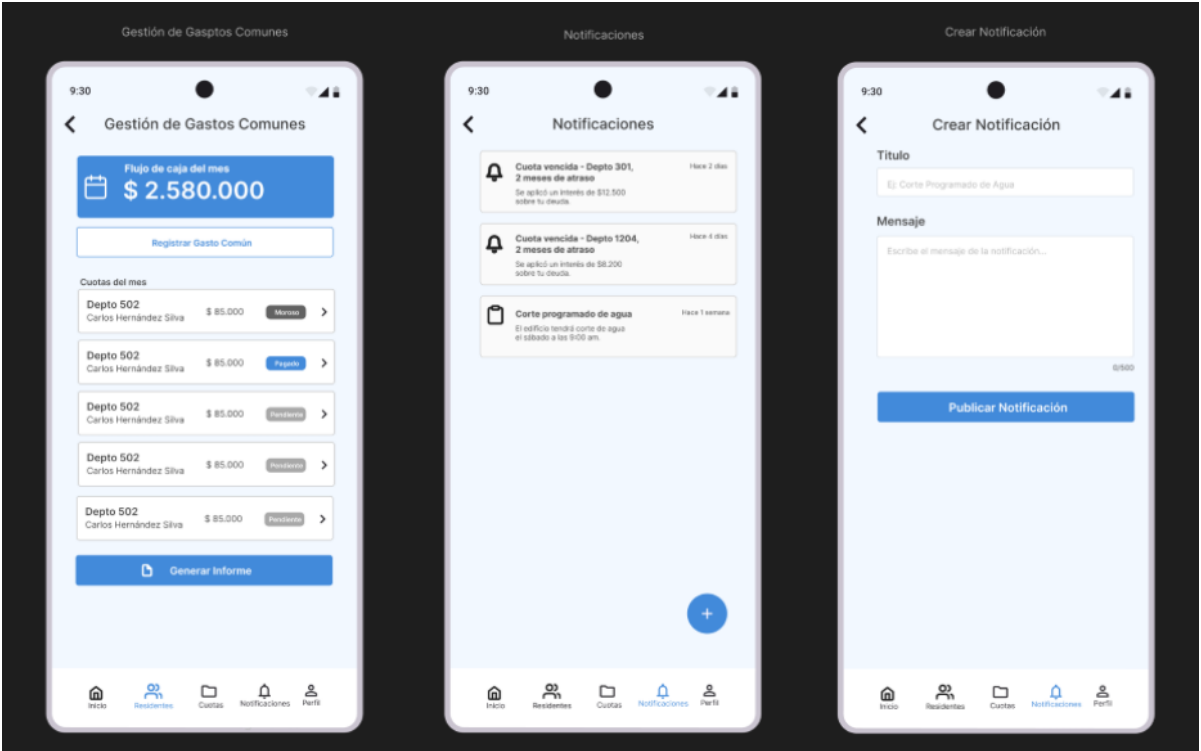
En esta etapa se incorporaron pantallas que no existían en baja fidelidad, para completar la experiencia de pago del Residente:

- **Detalle del gasto común, estado pendiente:** desglose por concepto (agua, luz, gas, mantención de ascensores, remuneraciones e insumos), monto total, fecha de vencimiento y botón "Pagar ahora".
- **Confirmación de pago:** mensaje de éxito, resumen del pago e indicación de que el comprobante fue enviado al correo del residente. El paso intermedio por la pasarela de pago es un servicio externo, por lo que no se diseñó como pantalla propia.
- **Detalle del gasto común, estado pagado:** mismo desglose, fecha de pago y botón "Descargar comprobante", sin opción de pagar nuevamente.
- **Perfil:** datos personales del residente.

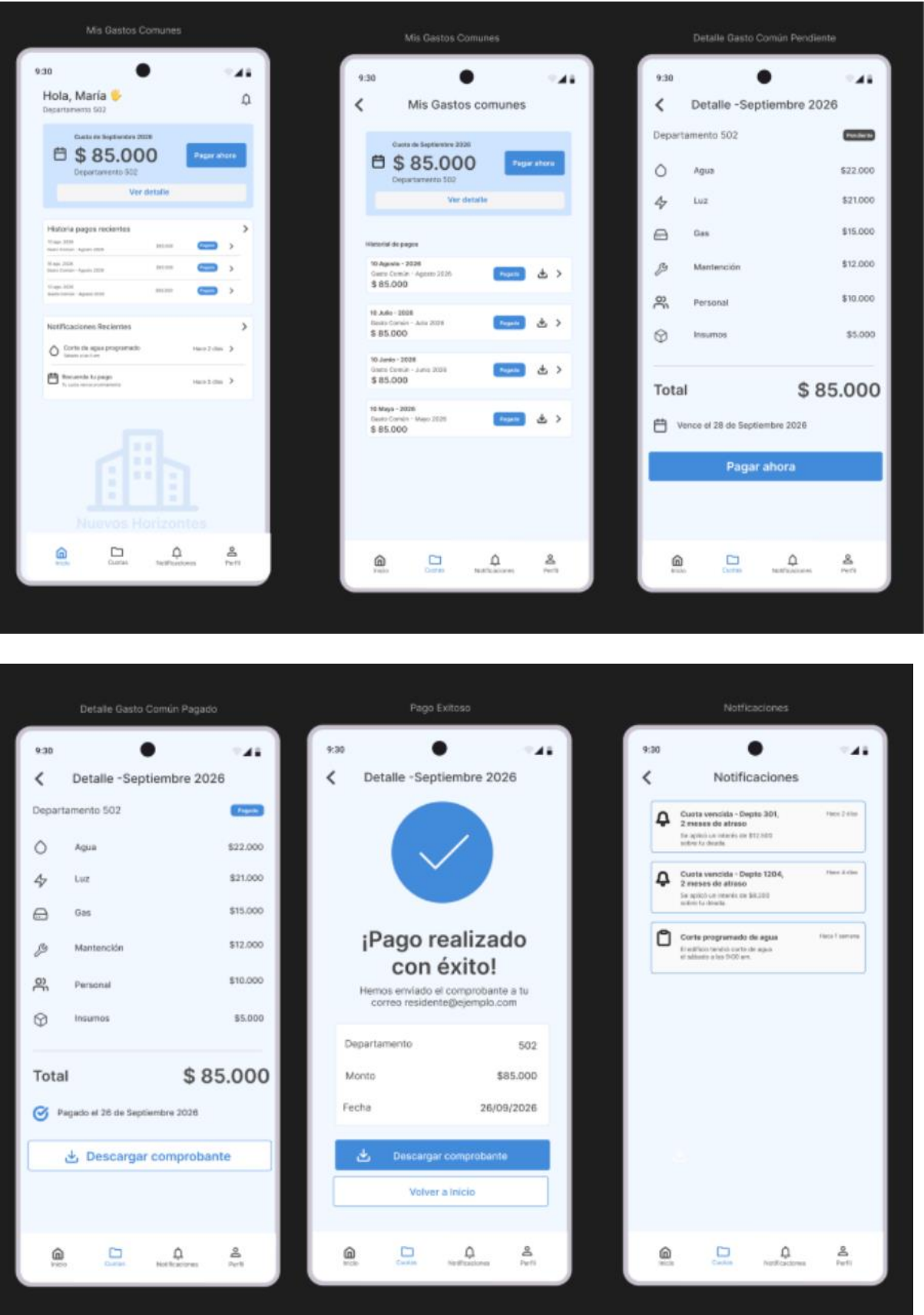
El flujo del Residente en alta fidelidad, con siete pantallas propias, es: Login (compartido), Mis gastos comunes, Detalle del gasto común, pasarela de pago externa, Confirmación de pago, Notificaciones y Perfil. El flujo del Administrador, con seis pantallas, es: Login (compartido), Registro de Residentes, Departamentos y Residentes, Gestión de Gastos Comunes, Notificaciones y Crear Notificación. La barra de navegación inferior se diferencia según el rol, mostrando a cada usuario solo las funciones que le corresponden.

Flujo Admin:





Flujo Residente:



Criterios de usabilidad y accesibilidad

El diseño se evaluó con el modelo de calidad ISO/IEC 25010, en particular con las características de usabilidad, eficiencia de desempeño, mantenibilidad y accesibilidad:

- **Usabilidad:** la navegación es consistente entre pantallas, con barra inferior fija. La retroalimentación es clara: la pantalla de confirmación informa el resultado del pago y los pasos siguientes. La minimización de errores se logra diferenciando el detalle según el estado de la cuota, de modo que nunca se ofrece pagar una cuota ya cancelada.
- **Eficiencia de desempeño:** las tareas frecuentes se resuelven en pocos pasos. La cuota pendiente se destaca al inicio de "Mis gastos comunes", y su detalle está a un toque de distancia.
- **Mantenibilidad:** el uso de componentes reutilizables permite modificar o ampliar la interfaz sin perder coherencia. Un mismo patrón de tarjeta con badge de estado se reutiliza en distintas pantallas.
- **Accesibilidad:** siguiendo los principios de percepción, operación, comprensión y robustez, la información de estado no depende solo del color, ya que cada badge (Pagado, Pendiente, Moroso) incluye texto. Los íconos van acompañados de etiquetas, y los mensajes y acciones utilizan lenguaje claro y directo.

Herramientas utilizadas en el prototipado

Ambos prototipos se desarrollaron en **Figma**, en su plan gratuito. Se eligió por sobre alternativas como Balsamiq, que es de pago, porque funciona en el navegador sin instalación, permite colaborar en tiempo real entre los integrantes de la dupla, ofrece componentes e íconos reutilizables que aseguran consistencia visual, y permite conectar las pantallas para simular la navegación. Es además una herramienta ampliamente usada en la industria del diseño de interfaces.

Enlace al prototipo:

<https://www.figma.com/design/OkXISuADP7uV2pOov4Fib0/lng.-de-Software?node-id=0-1&t=pQcg1JNjG7nIJFql-1>

Conclusiones

El diseño arquitectónico y el prototipado desarrollados para el sistema de gestión de pagos de gastos comunes del edificio Nuevos Horizontes permitieron responder a los objetivos planteados al inicio de este documento: automatizar el cálculo y cobro de cuotas, reducir la morosidad y entregar mayor transparencia a la administración y a los residentes.

La arquitectura serverless en AWS resultó coherente con el contexto real del cliente, sin equipo de TI dedicado y con un volumen de operaciones reducido, y permitió priorizar la lógica de negocio por sobre la administración de infraestructura. La aplicación del modelo de vistas 4+1 facilitó documentar el sistema desde múltiples perspectivas complementarias, validando que cada decisión de diseño respondiera a los requerimientos funcionales y no funcionales definidos.

El prototipado complementó este trabajo llevando la arquitectura a la experiencia del usuario. El prototipo de baja fidelidad, construido a partir de una sesión de brainstorming estructurado, permitió validar la estructura de las pantallas y las cuatro funcionalidades clave antes de invertir en diseño visual. El prototipo de alta fidelidad incorporó un sistema visual consistente para ambos roles del sistema, sobre un Login compartido: el Administrador, con seis pantallas de gestión, y el Residente, con siete pantallas propias, incluyendo el detalle diferenciado según el estado de la cuota, la confirmación de pago y el perfil, funcionalidades que la primera etapa no contemplaba. Su evaluación con el modelo ISO/IEC 25010 confirmó que decisiones como la retroalimentación clara, los estados que no dependen solo del color y los componentes reutilizables contribuyen a la usabilidad, la accesibilidad y la mantenibilidad del sistema.

El principal aprendizaje del proceso fue la importancia de cuestionar y validar cada decisión de diseño antes de darla por definitiva, evitando adoptar soluciones por defecto sin evaluar previamente su coherencia con el resto de la arquitectura. Como oportunidad de mejora, se identifica la necesidad de profundizar en la documentación de riesgos y buenas prácticas de seguridad, dado que el sistema maneja información sensible de los usuarios, así como la posibilidad de aprovechar el desacoplamiento logrado mediante funciones Lambda independientes por dominio para incorporar nuevas funcionalidades o escalar el sistema a otros edificios, sin modificar los módulos existentes.

Bibliografía y documentación relacionada

- Duoc UC. (2026). *Ingeniería de Software II – Guía de aprendizaje: Diseñando la solución de software. Experiencia 2.*
- Duoc UC. (2026). *Ingeniería de Software II – Elaborando diagramas del sistema informático, Parte I. Semana 3.*
- Duoc UC. (2026). *Ingeniería de Software II – Elaborando diagramas del sistema informático, Parte II. Semana 4.*
- Amazon Web Services. (s.f.). *Amazon DynamoDB Documentation*. AWS Documentation. <https://docs.aws.amazon.com/dynamodb/>
- Amazon Web Services. (s.f.). *Building Applications with Serverless Architectures*. AWS. <https://aws.amazon.com/lambda/serverless-architectures-learn-more/>
- Amazon Web Services. (s.f.). *Amazon API Gateway Documentation*. AWS Documentation. <https://docs.aws.amazon.com/apigateway/>
- Amazon Web Services. (s.f.). *Amazon Cognito Documentation*. AWS Documentation. <https://docs.aws.amazon.com/cognito/>
- Amazon Web Services. (s.f.). *Amazon Simple Email Service (SES) Documentation*. AWS Documentation. <https://docs.aws.amazon.com/ses/>

Anexo A — Especificación completa de casos de uso (nivel 2)

CU01 – Registrar departamento

Actor principal: Administrador. Objetivo: registrar un departamento del edificio. Precondición: sesión iniciada con permisos de gestión. Flujo principal: selecciona "Registrar departamento" → sistema solicita datos → administrador ingresa información → sistema valida → registra el departamento → confirma. Flujo alternativo: datos incompletos o inválidos, el sistema informa y solicita corrección. Postcondición: departamento registrado, disponible para asociar residente representante.

CU02 – Registrar residente

Actor principal: Administrador. Objetivo: registrar los datos de un residente, identificando si es el representante del departamento. Precondición: sesión iniciada, departamento registrado. Flujo principal: selecciona "Registrar residente" → sistema solicita datos incluyendo correo → administrador ingresa → sistema valida → registra al residente → continúa con CU03. Flujo alternativo: datos incompletos, el sistema informa el error. Postcondición: residente registrado. Relación: <<include>> CU03.

CU03 – Asociar residente a departamento

Actor principal: Administrador. Objetivo: asociar un residente al departamento correspondiente. Precondiciones: sesión iniciada, departamento y residente registrados. Flujo principal: selecciona residente → sistema muestra departamentos disponibles → selecciona departamento → sistema valida y vincula → confirma. Flujo alternativo: sin departamento válido, no se completa la operación. Postcondición: residente asociado al departamento.

CU04 – Actualizar residente

Actor principal: Administrador. Objetivo: actualizar la información del residente representante ante cambios en su vínculo con el departamento. Precondiciones: sesión iniciada, residente registrado. Flujo principal: selecciona gestión de residentes → sistema muestra registrados → selecciona residente → muestra datos actuales → modifica → sistema valida y actualiza

→ confirma. Flujo alternativo: información inválida, se mantienen los datos anteriores.

Postcondición: información actualizada.

CU05 – Dar de baja residente

Actor principal: Administrador. Objetivo: dar de baja a un residente representante sin vínculo vigente. Precondiciones: sesión iniciada, residente activo. Flujo principal: selecciona residente → selecciona "Dar de baja" → sistema solicita confirmación → confirma → sistema actualiza estado y revoca acceso → confirma. Flujo alternativo: cancelación de la operación, sin cambios. Postcondición: residente dado de baja, acceso revocado.

CU06 – Registrar gastos comunes

Actor principal: Administrador. Objetivo: registrar los gastos comunes del edificio. Precondiciones: sesión iniciada, permisos de gestión. Flujo principal: selecciona "Registrar gastos comunes" → sistema solicita información → administrador ingresa → sistema valida y registra → continúa con CU07 → confirma. Flujo alternativo: información incompleta, se solicita corrección. Postcondición: gastos registrados, disponibles para cálculo de cuotas. Relación: <<include>> CU07.

CU07 – Calcular cuotas

Actor asociado: Administrador. Ejecución: automática por el sistema, disparada al finalizar el registro de gastos comunes. Objetivo: calcular la cuota correspondiente a cada departamento según las reglas del negocio. Precondiciones: gastos comunes y departamentos registrados. Flujo principal: el sistema inicia automáticamente el cálculo al finalizar el registro de gastos comunes → obtiene los gastos registrados → identifica los departamentos → aplica las reglas de cálculo → determina la cuota de cada departamento → genera las nuevas cuotas → confirma el cálculo. Flujo alternativo: información insuficiente, no se generan las cuotas. Postcondición: cuota calculada por departamento. Relación: <<include>> CU08.

CU08 – Generar nueva cuota

Actor asociado: Administrador. Ejecución: automática por el sistema. Objetivo: registrar la nueva cuota calculada para cada departamento. Precondiciones: cálculo de cuotas realizado, departamento registrado. Flujo principal: recibe el resultado del cálculo → identifica el departamento → registra la nueva cuota → la deja disponible para consulta y pago → confirma la generación. Flujo alternativo: sin departamento válido, no se completa la

generación. Postcondición: nueva cuota registrada y disponible. Relación: <<extend>>
CU18 (condición: existe una cuota anterior impaga asociada al mismo departamento).

CU09 – Consultar gastos comunes pendientes

Actor principal: Residente Representante. Objetivo: consultar las cuotas pendientes de su departamento. Precondiciones: sesión iniciada, asociado a un departamento. Flujo principal: selecciona "Consultar gastos comunes pendientes" → sistema identifica el departamento → consulta cuotas pendientes → muestra montos. Flujo alternativo: sin cuotas pendientes, se informa. Postcondición: residente visualiza su estado actualizado.

CU10 – Realizar pago

Actor principal: Residente Representante. Actor secundario: Pasarela de pago. Objetivo: realizar el pago en línea de una cuota pendiente. Precondiciones: sesión iniciada, cuota pendiente existente. Flujo principal: consulta gastos pendientes → sistema muestra cuotas disponibles → selecciona cuota → sistema muestra monto → confirma → sistema envía a la pasarela de pago. Flujo alternativo: cancelación antes de confirmar, sin cambios en la cuota. Postcondición: solicitud de pago enviada. Relación: <<include>> CU11.

CU11 – Procesar pago

Actor principal: Pasarela de pago. Objetivo: procesar la transacción del pago. Precondiciones: pago confirmado por el residente, cuota seleccionada. Flujo principal: sistema envía solicitud → pasarela procesa la transacción → devuelve confirmación → sistema recibe confirmación → continúa con CU12. Flujo alternativo: transacción rechazada, se informa al residente y la cuota permanece pendiente. Postcondición: transacción confirmada o rechazada. Relación: <<include>> CU12.

CU12 – Registrar pago

Actor asociado: Residente Representante. Ejecución: automática tras confirmación de la pasarela. Objetivo: registrar el pago confirmado. Precondiciones: transacción confirmada, cuota pendiente. Flujo principal: recibe confirmación → identifica la cuota pagada → registra el pago en el historial → actualiza el estado de la cuota → continúa con CU13. Flujo alternativo: sin confirmación, no se registra el pago. Postcondición: pago registrado, cuota actualizada. Relación: <<include>> CU13.

CU13 – Emitir comprobante

Actor asociado: Residente Representante. Ejecución: automática tras el registro exitoso del

pago. Objetivo: generar un comprobante tras el pago exitoso. Precondiciones: pago confirmado y registrado. Flujo principal: obtiene la información del pago → genera el comprobante → lo asocia al pago → queda disponible en el historial → continúa con CU14. Flujo alternativo: sin pago confirmado, no se genera el comprobante. Postcondición: comprobante generado y disponible. Relación: <<include>> CU14.

CU14 – Enviar comprobante por correo electrónico

Actor principal: Servicio de correo electrónico. Objetivo: enviar el comprobante al correo del residente representante. Precondiciones: comprobante generado, correo registrado. Flujo principal: obtiene el correo → solicita el envío → el servicio procesa la solicitud → comprobante enviado. Flujo alternativo: envío no disponible, el comprobante permanece disponible para descarga. Postcondición: comprobante enviado por correo.

CU15 – Consultar historial de pagos

Actor principal: Residente Representante. Objetivo: consultar los gastos comunes pagados. Precondiciones: sesión iniciada, asociado a un departamento. Flujo principal: selecciona "Consultar historial de pagos" → sistema identifica el departamento → consulta pagos registrados → muestra historial → puede seleccionar un pago para ver su detalle. Flujo alternativo: sin pagos registrados, se informa. Postcondición: historial visualizado.

CU16 – Descargar comprobante de pago

Actor principal: Residente Representante. Objetivo: descargar el comprobante de un pago realizado. Precondiciones: sesión iniciada, pago con comprobante disponible. Flujo principal: consulta historial de pagos → selecciona un pago → sistema muestra el comprobante → selecciona "Descargar" → sistema entrega el archivo. Flujo alternativo: sin comprobante disponible, se informa. Postcondición: comprobante descargado.

CU17 – Consultar departamentos morosos

Actor principal: Administrador. Objetivo: consultar los departamentos con cuotas pendientes o vencidas. Precondiciones: sesión iniciada, cuotas registradas. Flujo principal: selecciona "Consultar departamentos morosos" → sistema identifica departamentos con cuotas vencidas → obtiene meses de atraso → obtiene los intereses previamente calculados y asociados a cada cuota vencida → continúa con CU19 → muestra el listado con el detalle de la deuda. Flujo alternativo: sin morosidad registrada, se informa. Postcondición: información de morosidad visualizada. Relación: <<include>> CU19.

CU18 – Calcular intereses

Actor asociado: Administrador. Ejecución: automática por el sistema, disparada al generarse una nueva cuota con deuda previa asociada. Objetivo: calcular los intereses de las cuotas vencidas. Precondiciones: debe haberse generado una nueva cuota con una cuota anterior vencida asociada al mismo departamento; tasa de interés vigente configurada. Flujo principal: identifica las cuotas vencidas del departamento → obtiene la tasa de interés vigente en la fecha de generación de la nueva cuota → determina los meses de atraso de cada cuota vencida → aplica esa tasa al monto de cada cuota vencida para calcular el interés del período → incorpora el interés al detalle de la deuda, sin modificar los intereses ya calculados en períodos anteriores. Flujo alternativo: sin tasa configurada, no se realiza el cálculo. Postcondición: monto de intereses del período calculado e incorporado a la deuda del departamento.

CU19 – Calcular total adeudado

Actor principal: Administrador. Objetivo: calcular el monto total adeudado por un departamento. Precondiciones: cuota vencida existente, intereses calculados. Flujo principal: obtiene cuotas pendientes → obtiene intereses generados → suma cuotas e intereses → calcula el total → muestra el detalle desglosado. Flujo alternativo: sin deuda vencida, se informa. Postcondición: total adeudado calculado y disponible.

CU20 – Configurar tasa de interés

Actor principal: Administrador. Objetivo: configurar la tasa de interés vigente para el cálculo de intereses. Precondiciones: sesión iniciada, permisos de gestión. Flujo principal: selecciona "Configurar tasa de interés" → sistema muestra la tasa actual → ingresa la nueva tasa → sistema valida → confirma → sistema registra la nueva tasa. Flujo alternativo: valor inválido, se solicita corrección. Postcondición: nueva tasa configurada.

CU21 – Enviar aviso de morosidad

Actor iniciador: Sistema, mediante proceso automático. Actor secundario: Servicio de correo electrónico. Objetivo: notificar al residente representante cuando su departamento presente cuotas morosas. Precondiciones: cuota vencida, residente representante con correo registrado. Flujo principal: identifica departamentos morosos → identifica al residente representante → genera el aviso → incorpora información de la deuda → solicita el envío por correo → el proceso se repite en cada mes de atraso. Flujo alternativo: sin correo válido, no

se completa el envío. Postcondición: aviso generado y enviado. Relación: <<include>> CU23.

CU22 – Publicar notificación administrativa

Actor principal: Administrador. Objetivo: publicar notificaciones administrativas generales dirigidas a los residentes. Precondiciones: sesión iniciada, permisos de gestión. Flujo principal: selecciona "Publicar notificación administrativa" → sistema solicita asunto y contenido → ingresa la información → sistema valida → confirma la publicación → sistema registra y publica dentro de la aplicación → solicita el envío por correo → confirma. Flujo alternativo: información incompleta, se solicita completarla. Postcondición: notificación publicada y disponible. Relación: <<include>> CU23.

CU23 – Enviar correo electrónico

Actor principal: Servicio de correo electrónico. Objetivo: enviar por correo electrónico los avisos y notificaciones generados por el sistema (morosidad, notificaciones administrativas, nuevas cuotas). Precondiciones: aviso o notificación generado, destinatarios con correo registrado. Flujo principal: obtiene el contenido y los destinatarios → entrega la solicitud al servicio de correo → el servicio procesa el envío → registra el envío realizado. Flujo alternativo: envío no disponible, la notificación permanece publicada dentro de la aplicación. Postcondición: correo enviado a los destinatarios registrados.

CU24 – Consultar notificaciones

Actor principal: Residente Representante. Objetivo: visualizar las notificaciones asociadas a su cuenta y las administrativas generales. Precondiciones: sesión iniciada, notificaciones disponibles. Flujo principal: selecciona "Consultar notificaciones" → sistema identifica al usuario → obtiene notificaciones de su cuenta y las generales → muestra el listado → selecciona una para ver el detalle. Flujo alternativo: sin notificaciones, se informa. Postcondición: notificaciones visualizadas.

CU25 – Generar informe de flujo de caja

Actor principal: Administrador. Objetivo: generar un informe del flujo de caja del edificio. Precondiciones: sesión iniciada, registros de ingresos y pagos existentes. Flujo principal: selecciona la opción → sistema solicita el período → selecciona período → sistema obtiene y procesa la información → genera el informe → lo muestra. Flujo alternativo: sin datos para el período, se informa. Postcondición: informe generado y disponible.

CU26 – Generar informe de pagos

Actor principal: Administrador. Objetivo: generar un informe detallado de los pagos registrados. Precondiciones: sesión iniciada, pagos registrados. Flujo principal: selecciona la opción → sistema solicita criterios → selecciona criterios → sistema obtiene y procesa los pagos → genera el informe → lo muestra. Flujo alternativo: sin pagos para los criterios, se informa. Postcondición: informe generado y disponible.

CU27 – Generar informe de morosidad

Actor principal: Administrador. Objetivo: generar un informe detallado de la morosidad. Precondiciones: sesión iniciada, información de cuotas y morosidad registrada. Flujo principal: selecciona la opción → sistema solicita criterios → selecciona → sistema identifica departamentos morosos y obtiene su información → genera el informe → lo muestra. Flujo alternativo: sin morosidad para los criterios, se informa. Postcondición: informe generado y disponible.

CU28 – Generar informe de departamentos

Actor principal: Administrador. Objetivo: generar un informe con información de la gestión de departamentos. Precondiciones: sesión iniciada, departamentos registrados. Flujo principal: selecciona la opción → sistema solicita criterios → selecciona → sistema obtiene y procesa la información → genera el informe → lo muestra. Flujo alternativo: sin información disponible, se informa. Postcondición: informe generado y disponible.

CU29 – Iniciar sesión

Actores principales: Administrador y Residente Representante. Objetivo: permitir el acceso al sistema mediante credenciales. Precondiciones: usuario registrado, credenciales vigentes. Flujo principal: accede a inicio de sesión → sistema solicita correo y contraseña → ingresa credenciales → sistema valida → identifica el rol → concede acceso a las funcionalidades correspondientes. Flujo alternativo: credenciales inválidas, se rechaza el acceso. Postcondición: usuario autenticado según su rol.

CU30 – Generar clave de primer ingreso

Actor asociado: Residente Representante. Objetivo: generar una clave aleatoria para el primer ingreso. Precondiciones: residente registrado, correo asociado. Flujo principal: identifica al nuevo residente representante → genera clave aleatoria → la asocia temporalmente a la cuenta → solicita el envío al correo → confirma la generación. Flujo

alternativo: sin correo registrado, no se completa el envío. Postcondición: clave temporal disponible. Relación: <<include>> CU31.

CU31 – Enviar clave por correo electrónico

Actor principal: Servicio de correo electrónico. Objetivo: enviar la clave de primer ingreso al correo registrado. Precondiciones: clave generada, correo registrado. Flujo principal: obtiene el correo → entrega la clave y el destinatario al servicio → el servicio procesa la solicitud → envía el mensaje → registra el envío. Flujo alternativo: envío no disponible, se informa que la entrega no pudo realizarse. Postcondición: clave enviada al correo del residente.

CU32 – Cambiar contraseña en primer ingreso

Actor principal: Residente Representante. Objetivo: exigir el cambio de la contraseña temporal en el primer inicio de sesión. Precondiciones: uso de la clave temporal, cuenta activa. Flujo principal: inicia sesión con la clave temporal → sistema identifica que es el primer ingreso → solicita nueva contraseña → ingresa → sistema valida → confirma el cambio → sistema actualiza la contraseña → permite continuar. Flujo alternativo: contraseña inválida, se solicita otra. Postcondición: clave temporal reemplazada por la definitiva.

CU33 – Rotar contraseña administrativa

Actor principal: Administrador. Objetivo: exigir la renovación de contraseña cada seis meses para el rol administrativo. Precondiciones: cuenta activa, seis meses transcurridos desde la última actualización. Flujo principal: intenta acceder → sistema verifica la fecha de última actualización → detecta que corresponde rotación → solicita nueva contraseña → ingresa → sistema valida y actualiza → permite continuar. Flujo alternativo: contraseña inválida, se solicita otra. Postcondición: contraseña actualizada, nuevo período de seis meses iniciado.

CU34 – Cerrar sesión

Actores principales: Administrador y Residente Representante. Objetivo: permitir finalizar manualmente la sesión activa. Precondición: usuario autenticado. Flujo principal: selecciona "Cerrar sesión" → sistema solicita finalizar la sesión → cierra la sesión → devuelve a la pantalla de acceso. Flujo alternativo: sin sesión activa, no se realiza ninguna operación. Postcondición: sesión finalizada.

CU35 – Notificar nueva cuota

Actor asociado: Residente Representante. Actor secundario: Servicio de correo electrónico. Objetivo: notificar automáticamente al residente cuando se genere una nueva cuota

disponible para pago. Precondiciones: nueva cuota generada, residente representante con correo registrado. Flujo principal: detecta la generación de la nueva cuota → identifica el departamento y al residente representante → genera la notificación → la publica dentro de la aplicación → solicita el envío por correo → confirma. Flujo alternativo: sin correo válido, la notificación permanece disponible solo dentro de la aplicación. Postcondición: notificación disponible en la aplicación y, cuando es posible, en el correo. Relación: <<include>> CU23.

Reservados todos los derechos Fundación Instituto Profesional Duoc UC. No se permite copiar, reproducir, reeditar, descargar, publicar, emitir, difundir, de forma total o parcial la presente obra, ni su incorporación a un sistema informático, ni su transmisión en cualquier forma o por cualquier medio (electrónico, mecánico, fotocopia, grabación u otros) sin autorización previa y por escrito de Fundación Instituto Profesional Duoc UC. La infracción de dichos derechos puede constituir un delito contra la propiedad intelectual.